CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

SOFTWARE TECHNOLOGY DEPARTMENT

REPORT

PROJECT: MAJOR RECOMMENDATION SYSTEM USING EXAM SCORES AND HOLLAND PERSONALITY TYPES

COURSE: PROJECT – BASIC TOPICS (CT239H)

MAJOR: SOFTWARE ENGINEERING (HIGH-QUALITY PROGRAM)

COHORT: 49

Advisor:

Ph.D. Phan Phương Lan

Student performing the project:

Full Name: Trần Quốc Tuấn Duy

Student ID: B2306662

SEMESTER III, ACADEMIC YEAR (2025–2026)

Can Tho, 08/2026

CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

SOFTWARE TECHNOLOGY DEPARTMENT

REPORT

PROJECT: MAJOR RECOMMENDATION SYSTEM USING EXAM SCORES AND HOLLAND PERSONALITY TYPES

COURSE: PROJECT – BASIC TOPICS (CT239H)

MAJOR: SOFTWARE ENGINEERING (HIGH-QUALITY PROGRAM)

COHORT: 49

Advisor:

Ph.D. Phan Phương Lan

Student performing the project:

Full Name: Trần Quốc Tuấn Duy

Student ID: B2306662

SEMESTER III, ACADEMIC YEAR (2025–2026)

Can Tho, 08/2026

ABSTRACT

Choosing an appropriate academic major is a critical decision that can significantly influence students' future career development. However, many students currently struggle with this process due to the lack of reliable guidance and the influence of excessive information from online sources. This project aims to develop a Major Recommendation System Using Exam Scores and Holland Personality Types to provide personalized recommendations based on students' academic performance and personality characteristics.

The proposed system integrates two complementary artificial intelligence approaches: Machine Learning Multi-class Classification models and a Hybrid Retrieval-Augmented Generation (Hybrid RAG) architecture using both Vector Database and Knowledge Graph technologies by Large Language Models (LLMs). Machine Learning Multi-class Classification model are utilized to analyze students' academic performance, subject combination scores, and Holland personality types (RIASEC) to identify suitable major groups. Meanwhile, the Hybrid RAG component enhances the system by retrieving relevant educational knowledge and generating personalized explanations, helping students understand why certain majors are recommended.

The results indicate that combining predictive models with generative AI can provide a more comprehensive career orientation experience. This project demonstrates the potential of AI in supporting educational decision-making through a personalized, data-driven, and explainable recommendation approach.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

No.	Term	Explain
1	RIASEC	Six vocational-interest types: Realistic, Investigative, Artistic, Social, Enterprising, and Conventional.
2	RAG	Retrieval-Augmented Generation; retrieves external evidence before an LLM generates an answer.
3	API	Application Programming Interface; a contract through which software components exchange requests and responses.
4	Frontend	The client-side interface through which users interact with the application.
5	Backend	The server-side services that handle validation, processing, model inference, retrieval, and APIs.
6	Hybrid RAG	A RAG approach that combines multiple retrieval methods, such as vector and graph search.
7	HyDE	Hypothetical Document Embeddings; generates a hypothetical answer and embeds it to improve retrieval.
8	BGE-M3	A multilingual, multifunctional, and multigranular text-embedding model used for retrieval.
9	Scikit-learn	A Python machine-learning library for preprocessing, training, evaluation, and model selection.
10	FastAPI	A Python web framework for building typed APIs with validation and automatic documentation.
11	React.js and Vite	React builds the user interface, while Vite provides the development server and build tool.
12	DaisyUI	A component library for Tailwind CSS that provides reusable user-interface elements.
13	ChromaDB	A vector database used to store embeddings, metadata, and similarity-search results.
14	Neo4j	A property-graph database that stores information as nodes, relationships, and properties.
15	Large Language Models (LLM)	Models trained on large datasets to process and generate natural-language content.
16	JSON/HTTP	JSON is the exchanged data format, while HTTP transports requests and responses over the web.
17	Rerank	A second-stage process that reorders retrieved candidates using a more precise relevance model.

18	CORS	Cross-Origin Resource Sharing; browser rules that control which origins may access an API.
19	UniPath	An external Holland assessment service linked by the application for users who need a RIASEC result. Author: Nguyễn Hoài Thi His LinkedIn: https://www.linkedin.com/in/nguyen-thi-1577951a7/
20	RRF	Reciprocal Rank Fusion; combines ranked result lists according to the reciprocal positions of candidates.
21	CSV	Comma-Separated Values; a plain-text format for storing tabular data.
22	Metadata	Descriptive information about data, such as a source URL, title, year, or chunk identifier.

CHAPTER 1: INTRODUCTION

1.1. Problem Statement

Choosing a university major is an important decision, but many students have difficulty identifying a suitable field because they may not fully understand their academic abilities, vocational interests, and personality characteristics. Decisions based mainly on social trends, income, employment opportunities, or advice from others may result in an unsuitable choice.

Therefore, this study proposes a major recommendation system that combines examination scores with Holland personality types (RIASEC). By considering both academic ability and vocational interests, the system provides personalized recommendations to support informed major selection. These recommendations serve only as career-orientation references and do not replace professional guidance or students' own judgment.

1.2. Objectives

1.2.1. General Objective

The primary objective of this project is to develop a major recommendation system that suggests suitable academic majors for students based on a combination of exam scores and Holland career personality types. The system provides personalized recommendations to help students better understand the compatibility between their academic abilities, career interests, and potential fields of study.

1.2.2. Specific Objectives

To achieve the general objective, the project focuses on the following specific objectives:

- Study the **Holland model (RIASEC)** and investigate the relationship between its six career personality types and different academic fields.
- Search for, collect, construct, and preprocess a suitable **training dataset** containing academic performance, subject combinations, Holland personality types, and corresponding major groups for the development of the classification model.
- Develop and train **Machine Learning Classification models** to predict suitable major groups based on students' exam scores, subject combinations, and Holland assessment results.
- Perform **model evaluation, comparison, and hyperparameter tuning** using appropriate evaluation metrics to identify and optimize the most suitable classification model for the recommendation task.

- Collect and process reliable information from relevant educational and career-oriented sources to construct a **domain-specific knowledge base** for the Retrieval-Augmented Generation (RAG) system.
- Develop a **Hybrid RAG architecture integrating vector-based retrieval and knowledge graph-based retrieval** to provide relevant contextual knowledge for the Large Language Model (LLM), enabling more informative and personalized explanations and recommendations.
- Design and develop a user interface that allows students to enter academic information, complete the Holland assessment, and view recommended majors with supporting explanations.
- Develop essential data management functions for academic majors, career-related knowledge, and other information required by the recommendation process.
- Conduct **system testing and evaluation** to assess functional correctness, usability, model performance, and the relevance of the generated recommendations.

1.3. Research Methodology

This study adopts an applied research approach involving literature review, data collection and preprocessing, requirement analysis, system development, and evaluation. Studies on major selection, Holland's RIASEC model, machine-learning classification, and recommendation systems are reviewed to establish the theoretical foundation. Examination scores, subject combinations, major information, and Holland–major relationships are then collected and standardized for system development.

The recommendation method uses examination scores and the user's dominant Holland type as inputs. The system calculates valid subject-combination scores and applies a trained classification model to recommend suitable major groups. A Hybrid RAG module provides detailed consultation using relevant information retrieved from vector and graph databases. Finally, the system is tested with different scenarios to evaluate its functionality, recommendation results, limitations, and usability.

1.4. Implementation Plan

ID	Tasks	Expected outcomes
1	Study the major recommendation problem, the Holland RIASEC model, Vietnamese university admission subject combinations, and machine learning classification methods. Define the project scope and prepare the report outline.	Theoretical foundation and project plan
2	Collect examination-score data, subject-combination information, Holland personality groups, and major-group information. Clean, standardize, and organize the collected data.	A structured training dataset
3	Analyze the dataset and construct a preprocessing pipeline, including missing-value handling, numerical feature scaling, and categorical feature encoding.	Data preprocessing pipeline
4	Train and compare Logistic Regression, Random Forest, Gradient Boosting, and Support Vector Machine models using cross-validation. Evaluate them using Accuracy, Precision, Recall, and F1-score.	Model comparison results
5	Select the Gradient Boosting model and optimize its hyperparameters using randomized search. Save the best-performing model for system integration.	Optimized classification model
6	Develop a RESTful API using FastAPI. Implement the calculation of admission subject combinations and the prediction of suitable major groups based on examination scores and Holland personality type.	Major-group prediction API
7	Develop the user interface using React.js and Vite. Implement score input, personality-type selection, recommendation result presentation and deploy the front end.	A web-based user interface accessible via the Internet.
8	Integrate the Frontend with the Backend.	Integrate the Recommendation Interface with the Backend

ID	Tasks	Expected outcomes
9	Collect and normalize detailed information about the academic majors offered by Can Tho University. Design the knowledge base and metadata structure for retrieval.	Major-information knowledge base
10	Study about RAG foundation and advanced. Develop the Hybrid RAG pipeline by combining vector search and graph search. Implement embedding, vector storage, knowledge-graph retrieval, query rewriting, HyDE, result fusion, and reranking.	Hybrid retrieval pipeline
11	Integrate Hybrid RAG into the recommendation system to provide detailed consultation on specific majors belonging to the predicted major groups. Test the complete workflow and resolve detected issues.	Complete integrated system
12	Evaluate the system and testing.	Testing results
13	Report writing and revision.	Final report

Table 1.1. Thirteen-week implementation plan

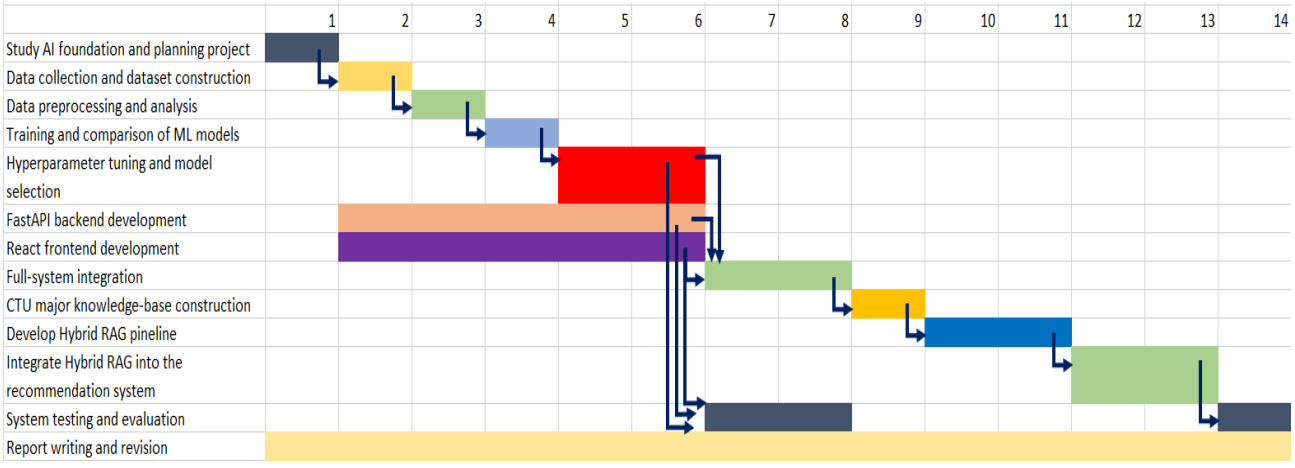


Figure 1.1. Gantt chart for the implementation plan and task dependencies

Project Documentation Folder *(include Project Plan, Materials, Experiment Results):*

https://drive.google.com/drive/folders/1qq1npU0-gsDBgcHELsw3boBUi1ASctkm?usp=drive_link

CHAPTER 2: THEORETICAL FOUNDATION

2.1. Holland RIASEC Model

Holland's RIASEC[1] model is a vocational-interest framework developed to describe the relationship between individuals and educational or occupational environments. The model classifies vocational interests into six categories: Realistic, Investigative, Artistic, Social, Enterprising, and Conventional.

The six RIASEC types are summarized as follows:

Type	Examples of Fields	Typical Traits
Realistic	computer engineering, forestry, surveying, poultry science, mining technology, computer installation, heating/AC technician, animal training, pharmacy technician, massage, meat cutter, carpentry, turf management, furniture design	mechanical and athletic abilities, likes to work outdoors and with tools and machines; might be described as conforming, frank, hardheaded, honest, humble, materialistic, natural, normal, persistent, practical, shy, thrifty
Investigative	biology, chemistry, physics, geology, anthropology, laboratory assistant, medical technician, social psychology, computer science, pharmacy, criminology, geography, general studies, liberal arts, psychology	math and science abilities, likes to work alone and to solve problems; might be described as analytical, complex, critical, curious, independent, intellectual, introverted, pessimistic, precise, rational
Artistic	composer, music, stage director, dance, interior decoration, acting, writing, drawing, languages, painting, speech, philosophy, comparative literature, industrial design, landscape architecture, historic preservation, housing studies, journalism	artistic skills, enjoys creating original work, has a good imagination; may be described as complicated, disorderly, emotional, idealistic, imaginative, impulsive, independent, introspective, nonconforming, original
Social	education, speech therapy, counseling, clinical psychology, nursing, dental hygiene, sports medicine, ministry/theology, music therapy, special education, home health, food and nutrition	likes to help, teach, and counsel people; may be described as cooperative, friendly, generous, helpful, idealistic, kind, responsible, sympathetic, tactful, understanding, warm
Enterprising	marketing, television production, business, sales, hospitality management, sports administration, urban planning, acting/ directing, advertising, entrepreneurship, educational administration, financial planning, pre-law, insurance, political science, real estate	leadership and public speaking abilities, is interested in money and politics, likes to influence people; described as acquisitive, agreeable, ambitious, attention getting, domineering, energetic, extroverted, impulsive, optimistic, self-confident, sociable
Conventional	bookkeeping, accounting, office management, court reporting, desktop publishing, medical laboratory assisting, computer operator, hematology technology, business communications	clerical and math abilities, likes to work indoors and to organize things; described as conforming, careful, efficient, obedient, orderly, persistent, practical, thrifty, unimaginative

Table 2.1. Holland RIASEC types, example fields, and typical traits

In this project, the dominant Holland type is combined with examination scores and subject-combination information to recommend potentially suitable major groups. It is used as one input factor and does not replace professional educational or career counselling.

2.2. Machine Learning

2.2.1. Multi-class Classification

Multi-class classification[2] is a supervised machine learning task in which each input instance is assigned to one class among more than two possible classes. In the proposed system, the input consists of the subject-combination code, the corresponding examination score, and the student's dominant Holland personality type. The target variable represents the suitable major group. Instead of using only the predicted class, the probability distribution produced by the classifier can be used to rank multiple major groups. This approach allows the system to provide several recommendations rather than presenting only one fixed result.

2.2.2. Classification Algorithms

Four supervised learning algorithms were investigated for the major-group classification task: Logistic Regression, Random Forest, Gradient Boosting, and Support Vector Machine. These algorithms represent different learning approaches, including linear classification, bagging, boosting, and maximum-margin classification. [8]

2.2.2.1. Logistic Regression

Logistic Regression[3] is a linear classification algorithm that estimates the probability that an input belongs to each possible class. Despite its name, it is used for classification rather than predicting a continuous value. In this project, it is used as a baseline because it is simple, computationally efficient, and suitable for evaluating whether the relationships among examination scores, subject combinations, Holland types, and major groups can be represented linearly.

2.2.2.2. Random Forest

Random Forest[4] is an ensemble learning algorithm that combines multiple decision trees. During training, each tree is constructed from a bootstrap sample drawn from the original training data. In addition, only a random subset of features is considered at each split. These randomization mechanisms create trees with different structures and reduce the dependence on any individual tree. It is suitable for this project because it can capture nonlinear relationships and interactions among examination scores, subject combinations, and Holland types while reducing the overfitting associated with a single decision tree.

2.2.2.3. Gradient Boosting

Gradient Boosting[5] is another tree-based ensemble method. Unlike Random Forest, which trains trees relatively independently and aggregates their predictions, Gradient Boosting constructs trees sequentially. Each new tree is trained to reduce the errors remaining from the current ensemble. It is suitable for this project because the relationship between student information and major groups may be complex and nonlinear. The algorithm is also effective for structured tabular data such as the dataset used in this system.

2.2.2.4. Support Vector Machine

Support Vector Machine[6] is a classification algorithm that searches for a decision boundary with the largest possible margin between classes. The training observations closest to the boundary are called support vectors because they determine the position of the separating hyperplane. It is suitable for this project because it can distinguish multiple major groups based on combinations of numerical and encoded categorical features, particularly for datasets of moderate size.

2.2.3 Model Evaluation Methods

To evaluate and compare the classification algorithms, this project uses five-fold stratified cross-validation together with Accuracy, Precision, Recall, F1-score, a confusion matrix, Mean Squared Error (MSE), and Mean Absolute Error (MAE). These measures describe both class-label performance and the quality of the predicted class-probability vectors.

2.2.3.1 K-fold Cross-validation

K-fold [7] cross-validation divides the dataset into *K* subsets. The model is trained on *K-1* subsets and evaluated on the remaining subset. This process is repeated *K* times so that every subset is used once for evaluation.

In this project, five-fold stratified cross-validation is used. Stratification maintains approximately the same class distribution in every fold. The mean and standard deviation of the evaluation scores are reported to compare model performance and stability.

2.2.3.2 Accuracy

Accuracy represents the proportion of correctly classified samples among all evaluated samples.

$$Accuracy = \frac{Number\ of\ correct\ predictions}{Total\ number\ of\ predictions} \quad (1)$$

Accuracy provides an overall measure of model performance. However, it should be considered together with other metrics, especially when the class distribution is imbalanced.

2.2.3.3 Precision

Precision measures the proportion of samples predicted as a class that actually belong to that class.

$$Precision = \frac{TP}{TP+FP} \tag{2}$$

High Precision indicates that the model produces relatively few false-positive predictions.

2.2.3.4 Recall

Recall measures the proportion of samples belonging to a class that are correctly identified by the model.

$$Recall = \frac{TP}{TP+FN} \tag{3}$$

High Recall indicates that the model misses relatively few samples of the corresponding class.

2.2.3.5 F1-score

The F1-score is the harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision+Recall} \tag{4}$$

The F1-score provides a balanced evaluation when both false-positive and false-negative predictions are important.

Because this project involves multiple major-group classes, weighted Precision, weighted Recall, and weighted F1-score are used. The weighted average calculates each class metric separately and assigns a weight according to the number of samples in that class:

$$Metric_{weighted} = \sum_{k=1}^{K} \left(\frac{n_k}{N}\right) Metric_k \quad (5)$$

where n_k is the number of samples in class k, N is the total number of samples, and $Metric_k$ is the score of class k.

2.2.3.6 Confusion Matrix

A confusion matrix presents the number of correct and incorrect predictions for each class. Its rows normally represent actual classes, while its columns represent predicted classes. It helps identify which major groups are correctly recognized and which groups are frequently confused with one another.

2.2.3.7 Mean Squared Error and Mean Absolute Error

For multi-class classification, let N be the number of samples, C the number of classes, y_ic the one-hot encoded true value for sample i and class c, and p_ic the predicted probability for the same class. MSE and MAE are calculated over all sample-class pairs as follows:

$$MSE = \frac{1}{NC} \sum_{i=1}^{N} \sum_{c=1}^{C} (y_{ic} - p_{ic})^2 \quad (6)$$

$$MAE = \frac{1}{NC} \sum_{i=1}^{N} \sum_{c=1}^{C} |y_{ic} - p_{ic}| \quad (7)$$

MSE gives greater weight to large probability errors because the differences are squared, whereas MAE represents the average absolute probability deviation. Lower values are better. In this project, they supplement Accuracy and weighted F1-score and must be computed from probability vectors, not from arbitrary numeric codes assigned to the major-group labels.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation[9] [10] is an approach that combines information retrieval with a generative language model. Before generating an answer, the system retrieves relevant information from an external knowledge base and provides it as additional context to the language model. This mechanism allows the generated response to be grounded in domain-specific information instead of relying entirely on the model's internal knowledge.,

In the proposed system, RAG is used after the machine learning model predicts suitable major groups. Its purpose is to retrieve detailed information about majors offered by Can Tho University[31] and generate a more informative consultation response for the user.

2.3.1 Text Embedding and Vector Search

Text embedding is the process of converting text into numerical vectors that represent its semantic meaning. Before embedding, documents are divided into smaller chunks. Both document chunks and user queries are then converted into vectors using the BGE-M3[13] embedding model. Texts with similar meanings are expected to have vectors located close to each other in the embedding space.

2.3.2 Knowledge Graph and Graph Search

A knowledge graph represents information as nodes, relationships, and properties. In this project, nodes describe entities such as majors, major groups, subject combinations, and Holland types. Relationships represent explicit connections between these entities.

2.3.3 Hybrid Retrieval and Reranking

Hybrid retrieval combines vector search and graph search to use both semantic similarity and explicit relationships. The user query is first rewritten and expanded using HyDE. Vector search and graph search are then performed independently, and their ranked results are merged using Reciprocal Rank Fusion. [12], [14], [15]

2.3.4 Hybrid RAG Retrieval Evaluation Metrics

Retrieval evaluation checks whether the correct source documents are returned before answer generation. Let Q be the number of answerable test queries, R_q the set of relevant documents for query q, and D_q^k the top-k retrieved documents. Out-of-scope cases without an expected source are evaluated separately and are not included in Q for the retrieval metrics.

Hit Rate@k is the proportion of queries for which at least one relevant document appears in the top-k results. It measures retrieval coverage.

$$Hit\ Rate@k\ = \frac{1}{Q}\sum_{q=1}^{Q}\ I\left(R_q \cap\ D_q^k \neq\ \emptyset\right) \tag{8}$$

Mean Reciprocal Rank (MRR@k) rewards systems that place the first relevant document near the beginning of the ranked list, where rank_q is its position.

$$MRR@k\ = \frac{1}{Q}\sum_{q=1}^{Q}\ \left(\frac{1}{rank_q}\right) \tag{9}$$

Precision@k measures the proportion of the top-k results that are relevant, while Recall@k measures how much of the known relevant-document set is retrieved.

$$Precision@k = \frac{1}{Q}\sum_{q=1}^{Q}\left(\frac{|R_q \cap D_q^k|}{k}\right) \quad (10)$$

$$Recall@k = \frac{1}{Q}\sum_{q=1}^{Q}\left(\frac{|R_q \cap D_q^k|}{|R_q|}\right) \quad (11)$$

Normalized Discounted Cumulative Gain (nDCG@k) evaluates ranking quality by giving greater weight to relevant documents at higher positions. DCG uses a logarithmic rank discount, and IDCG is the best possible DCG for the query.

$$nDCG@k = \frac{1}{Q}\sum_{q=1}^{Q}\left(\frac{DCG_q^k}{IDCG_q^k}\right) \quad (12)$$

Mean retrieval time reports efficiency, where t_q is the elapsed retrieval time for query q. Lower time indicates faster retrieval, but it should be interpreted together with effectiveness metrics.

$$T_{mean} = \frac{1}{Q}\sum_{q=1}^{Q} t_q \quad (13)$$

Together, these metrics distinguish source coverage, first-result quality, top-k relevance, ranking quality, and computational cost. They evaluate the retrieval stage only; generated-answer correctness and faithfulness require separate generation evaluation.

2.4 Supporting Technologies

2.4.1 Python and scikit-learn

Python is used as the primary backend and machine learning programming language. Scikit-learn[11] provides preprocessing pipelines, classification algorithms, cross-validation, model evaluation, and hyperparameter optimization. It was selected because its components can be combined into a consistent training and prediction pipeline.

2.4.2 FastAPI

FastAPI[19][20] is used to develop the RESTful backend services. It receives examination scores and Holland personality information, invokes the trained machine learning model, and exposes the recommendation and consultation functions to the frontend.

2.4.3 React.js and Vite[17][18]

React.js is used to develop the interactive user interface, while Vite provides the frontend development and build environment. The interface supports score input, Holland personality selection, recommendation presentation, and interaction with the consultation component.

2.4.4 ChromaDB[24] and Neo4j[16][25]

ChromaDB is used to store document embeddings and support semantic vector retrieval. Neo4j is used to represent entities and relationships in the major knowledge graph. The combination allows the system to retrieve both semantically similar content and structurally related information.

2.4.5 BGE-M3[13] and Large Language Models

BGE-M3 is used to generate multilingual text embeddings for the retrieval process. A large language model is used to rewrite queries, generate hypothetical documents, and produce consultation responses based on the retrieved contexts. The system supports configurable language-model providers, including Gemini 3.1 Flash-Lite[23] and GPT 5.6 Sol[21] [22].

CHAPTER 3: APPLICATION RESULTS

3.1. Software Requirement Specification

This section describes the requirements of the proposed major recommendation system. The system combines examination scores, the dominant Holland vocational-interest type, a machine learning classifier, and Hybrid RAG to recommend major groups and provide detailed consultation on majors offered by Can Tho University.

3.1.1. Overall Description

3.1.1.1. Product Perspective

The proposed system is a web-based decision-support application for Vietnamese high-school students. It combines examination scores, valid university admission subject combinations, and the student's dominant Holland RIASEC type to recommend suitable major groups. After a group is selected, a Hybrid Retrieval-Augmented Generation (Hybrid RAG) module provides a more detailed consultation grounded in the project's curated university knowledge sources. The system supports career exploration; it does not replace official admission information or professional counselling.

3.1.1.2. Product Functions

The system allows a student to enter subject scores and a Holland type, automatically derives valid subject combinations, predicts and ranks three suitable major groups for each combination, and displays their confidence scores. The student can then select a group, provide a short personal description, receive a grounded explanation, ask follow-up questions, and open the cited source links.

3.1.1.3. User Characteristics

The primary user is a high-school student who can use a modern web browser and understands the 0–10 Vietnamese examination-score scale. No knowledge of machine learning is required. The user is expected to know a dominant Holland type obtained from a RIASEC assessment or to use the external assessment link provided by the interface.

3.1.1.4. Operating Environment

The client runs in a modern desktop or mobile browser. The frontend is implemented with React.js and Vite and communicates through JSON/HTTP with a Python FastAPI backend. The prediction module loads a trained scikit-learn model. The consultation module depends on a Chroma vector store, a Neo4j knowledge graph, embedding and reranking models, and an available large language model provider.

3.1.1.5. Constraints, Assumptions, and Dependencies

Scores must be numeric values from 0 to 10; Mathematics and Literature are required, while two optional subjects must be different. Recommendations are limited by the subject-combination rules, labels represented in the training dataset, and the quality of the curated knowledge base. The consultation function requires configured external services and network connectivity. Admission regulations and university information may change; therefore, users must verify important decisions against current official sources.

3.1.2. Specific Requirements

3.1.2.1. Input Requirements

Input	Type and validation	Purpose
Mathematics and Literature scores	Required decimal values in [0, 10]	Generate eligible combinations and calculate combination scores
Two optional subjects and scores	Required; subjects must be distinct; scores in [0, 10]	Generate eligible combinations from the selected subjects
Holland type	One value from R, I, A, S, E, or C	Represent the student's dominant vocational-interest type
Selected major group	One group returned by the prediction step	Constrain the detailed consultation topic
Personal description	User-entered text	Personalize retrieval and the generated explanation
Follow-up question and recent history	Text; recent turns only	Maintain conversational context without sending an unlimited history

Table 3.1. Input requirements

3.1.2.2. Processing Requirements

The backend validates user input, calculates valid subject-combination scores, and uses the trained model to return the three major groups with the highest probabilities. For consultation, it rewrites the query, performs Hybrid RAG retrieval from vector and graph databases, fuses and reranks the results, and sends the most relevant contexts to the language model.

3.1.2.3. Output Requirements

Prediction output shall contain the subject-combination code, total score, component subjects, and the three ranked major-group labels with their probability values. Consultation output shall present the major name or group, a concise description, reasons for suitability, the relation to the selected Holland type, orientation advice, suggested next questions, and official reference links when available. If reliable context is insufficient, the response shall state the limitation instead of inventing information.

3.1.2.4. External Interface Requirements

Interface	Request	Response
POST /predict	subjects, scores, holland	Valid combinations and Top-3 major groups for each combination
POST /chat	group_major, describe, recent history/question	Grounded consultation text and reference sources
React user interface	Forms, selection controls, and chat input	Ranked results, consultation dashboard, loading and error states

Table 3.2. External interface requirements

3.1.2.5. Data Requirements

The prediction dataset stores subject-combination code, combination score, Holland type, and target major-group label. The runtime also uses the official subject-combination mapping and serialized trained models. The RAG knowledge base stores textual chunks and metadata in Chroma and explicit entities and relationships in Neo4j. Source metadata accepted for user-facing citations is restricted to the configured official Can Tho University admission domain.

3.1.3. Functional Requirement

Figure 3.1 presents the student's interactions with the system. The student enters examination information and a Holland type to receive recommendations, selects one recommended major group, requests detailed consultation, asks follow-up questions, and opens the available reference sources.

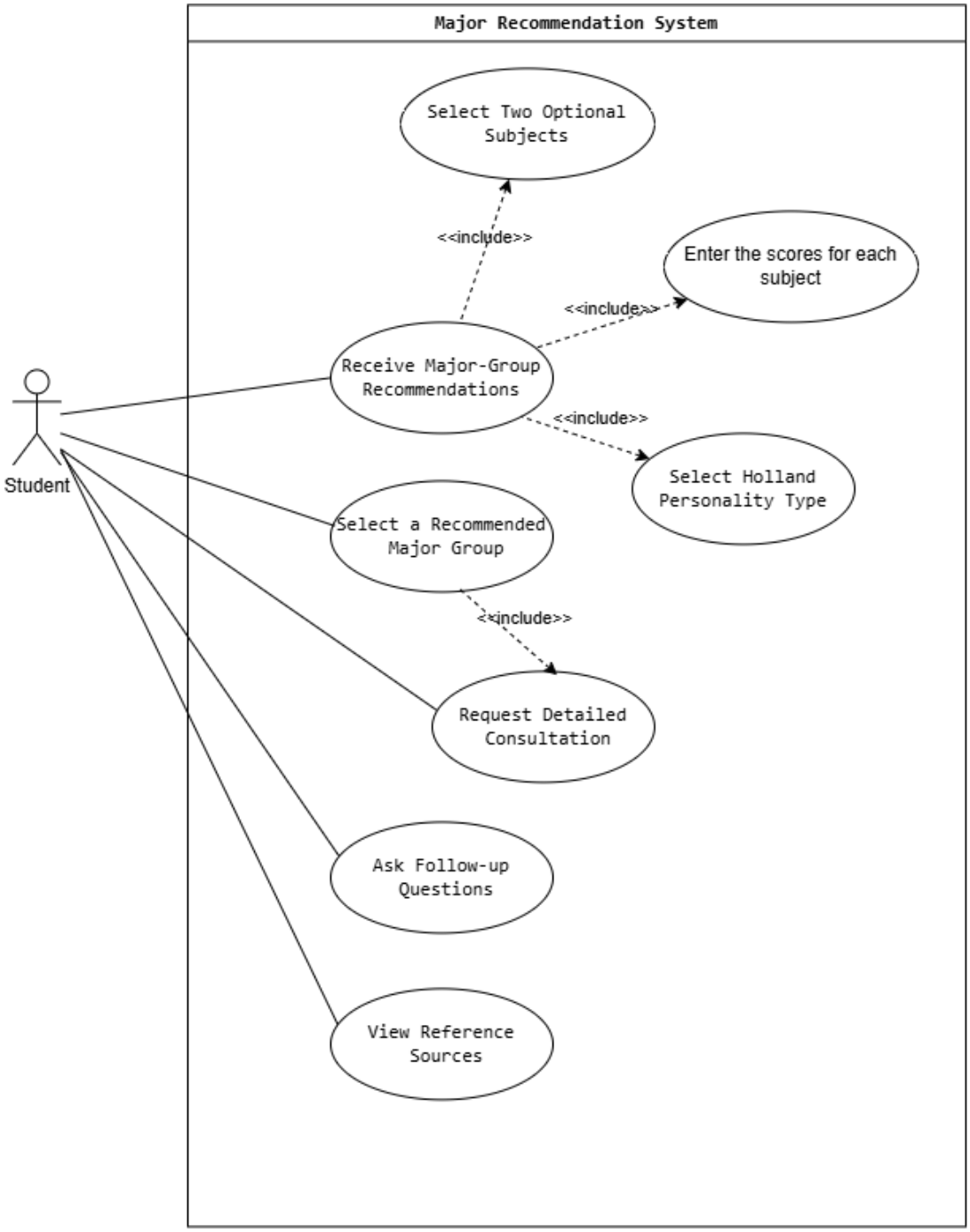


Figure 3.1. Overall use-case diagram

3.1.3.1. Functional Requirement List

ID	Function	Requirement	Priority
FR-01	Enter examination data	The student shall enter Mathematics, Literature, two distinct optional subjects, and scores from 0 to 10.	High
FR-02	Select Holland type	The student shall select exactly one RIASEC type; the interface shall provide a link to an external assessment.	High
FR-03	Validate input	The system shall reject missing, duplicated, nonnumeric, or out-of-range input and show a clear message.	High
FR-04	Generate combinations	The system shall identify all supported admission combinations that can be formed from the entered subjects.	High
FR-05	Calculate scores	The system shall calculate the total score for each valid subject combination.	High
FR-06	Predict major groups	The system shall obtain class probabilities from the trained classification model.	High
FR-07	Rank results	The system shall display the three highest-probability major groups for every valid combination.	High
FR-08	Select recommendation	The student shall select one predicted group for detailed consultation.	High
FR-09	Describe personal goals	The student shall provide an optional description of interests, strengths, and career expectations.	Medium
FR-10	Retrieve hybrid evidence	The system shall retrieve relevant vector and graph evidence, fuse the rankings, and rerank the candidates.	High
FR-11	Generate consultation	The system shall generate an explanation grounded in retrieved contexts and the selected group.	High
FR-12	Support follow-up questions	The student shall ask follow-up questions using recent conversation context.	Medium

ID	Function	Requirement	Priority
FR-13	Show references	The interface shall display available official source links associated with the answer.	High
FR-14	Restart workflow	The student shall return to previous steps or restart the recommendation process.	Low

Table 3.3. Functional requirements

3.1.3.2. Use Case UC-01: Obtain Major-Group Recommendations

Field	Description
Actor	Student
Goal	Obtain ranked major-group recommendations from examination scores and Holland type.
Preconditions	The frontend and prediction API are available; the classification model has been loaded.
Trigger	The student submits the completed score and Holland form.
Normal flow	1. The system validates all inputs. 2. It generates valid subject combinations. 3. It calculates each combination score. 4. The model returns class probabilities. 5. The system sorts the probabilities and displays the Top-3 groups for each combination.
Alternative/exception flows	Invalid input: show the corresponding validation message. No supported combination: inform the student and request another subject selection. Backend/model error: show a recoverable error without exposing internal details.
Postconditions	The student can select one recommended major group for consultation.

Table 3.4. Use case UC-01: Obtain major-group recommendations

3.1.3.3. Use Case UC-02: Select a Recommended Major Group

Field	Description
Actor	Student

Field	Description
Goal	Select one recommended major group as the scope of the detailed consultation.
Preconditions	At least one prediction result has been displayed.
Trigger	The student selects a major-group result card.
Normal flow	1. The student reviews the ranked recommendations. 2. The student selects one major group. 3. The interface stores the selected group and enables the detailed-consultation step.
Alternative/exception flows	No recommendation selected: the consultation action remains disabled and the student is prompted to select a group.
Postconditions	The selected major group becomes the consultation scope.

Table 3.5. Use case UC-02: Select a recommended major group

3.1.3.4. Use Case UC-03: Request Detailed Consultation

Field	Description
Actor	Student
Goal	Receive a personalized and evidence-grounded explanation of a selected major group.
Preconditions	A predicted major group has been selected; the RAG services and language model are configured.
Trigger	The student submits a personal description for the selected group.
Normal flow	1. The backend rewrites the request and generates HyDE text. 2. Vector and graph retrieval are executed. 3. Candidate rankings are fused and reranked. 4. Relevant contexts are placed in the prompt. 5. The language model returns structured consultation content and references.
Alternative/exception flows	Insufficient evidence: return a limitation notice. Unavailable retrieval or model service: return a controlled error and allow retry. Invalid source metadata: skip the link from user-facing references.

Field	Description
Postconditions	The initial consultation and available sources are shown; follow-up chat is enabled.

Table 3.6. Use case UC-03: Request detailed consultation

3.1.3.5. Use Case UC-04: Ask a Follow-up Question

Field	Description
Actor	Student
Goal	Clarify the recommendation through a contextual follow-up question.
Preconditions	An initial consultation has been produced.
Trigger	The student sends a follow-up question.
Normal flow	1. The interface sends the question with a bounded recent history. 2. The backend retrieves relevant evidence. 3. The language model generates a direct grounded answer. 4. The interface appends the answer and references to the conversation.
Alternative/exception flows	Empty question: do not submit. Service failure: preserve the conversation and allow retry. Insufficient evidence: disclose the limitation.
Postconditions	The conversation contains the new question and answer without changing the original prediction result.

Table 3.7. Use case UC-04: Ask a follow-up question

3.1.3.6. Use Case UC-05: View Reference Sources

Field	Description
Actor	Student
Goal	Open the official information sources used to support a consultation response.
Preconditions	A consultation response containing at least one accepted reference URL has been displayed.
Trigger	The student selects a reference link.

Field	Description
Normal flow	1. The system displays the source title and URL with the consultation. 2. The student selects a link. 3. The browser opens the official source in a new page or tab.
Alternative/exception flows	No accepted source is available: the reference section is omitted or a limitation is shown. Invalid or unavailable URL: the browser reports that the source cannot be opened.
Postconditions	The student can verify the consultation against the external official source.

Table 3.8. Use case UC-05: View reference sources

3.1.4. Nonfunctional Requirements

The following requirements follow the categories used in the supplied nonfunctional-requirement guideline. Numerical values are acceptance targets for validation; they are not reported as achieved unless a corresponding test result is recorded in Section 3.3.

ID	Category	Requirement	Verification
NFR-01	Performance	After the prediction model is loaded, 95% of /predict requests should complete within 3 seconds under the target demonstration workload.	Measure API latency over a representative request set.
NFR-02	Performance	The interface should acknowledge a consultation request immediately with a loading state; 95% of /chat requests should complete within 40 seconds when dependencies are available.	Measure end-to-end response time and observe loading feedback.
NFR-03	Usability	A first-time student shall be able to complete the three-step workflow without technical knowledge; fields, scores, errors, ranks, and sources shall have visible labels.	Task-based usability review with representative users.

ID	Category	Requirement	Verification
NFR-04	Reliability	Invalid input or an unavailable dependency shall produce a controlled message and shall not terminate the frontend or backend process.	Negative and dependency-failure tests.
NFR-05	Reliability	When the knowledge base does not provide adequate evidence, the consultation shall explicitly state the limitation rather than fabricate a definitive answer.	Test with out-of-scope and low-evidence questions.

Table 3.9. Nonfunctional requirements

3.2. Software Design

3.2.1. Application Architecture

The application follows a client–server architecture. The React-based web client allows users to enter examination scores, select a Holland type, view recommended major groups, and use the consultation chatbot. It communicates with the backend through RESTful APIs using JSON and provides a link to the external UniPath[33] Holland assessment.

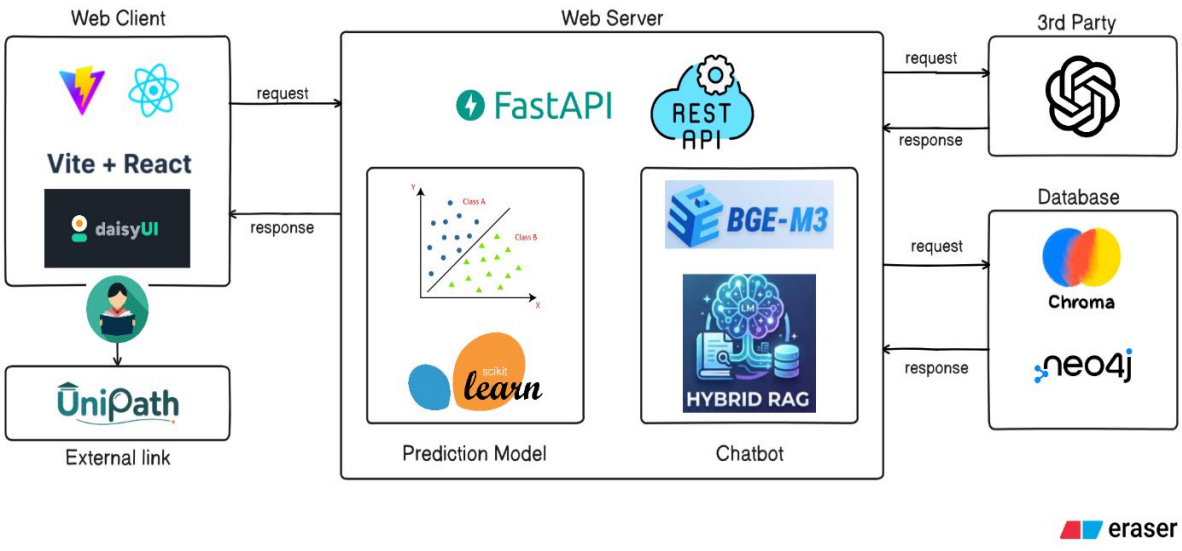


Figure 3.2. Overall system architecture

The prediction module applies predefined rules to calculate valid subject-combination scores before using the trained scikit-learn model to predict suitable major groups. Meanwhile, the consultation module rewrites the query, performs HyDE, retrieves information from both Chroma and Neo4j, combines the results using RRF, and reranks them before sending the most

relevant contexts to the language model. This separation ensures that the language model supports consultation without changing score calculations or classification results. Python and FastAPI were selected to integrate the machine-learning and retrieval components, while React communicates with the backend through APIs.

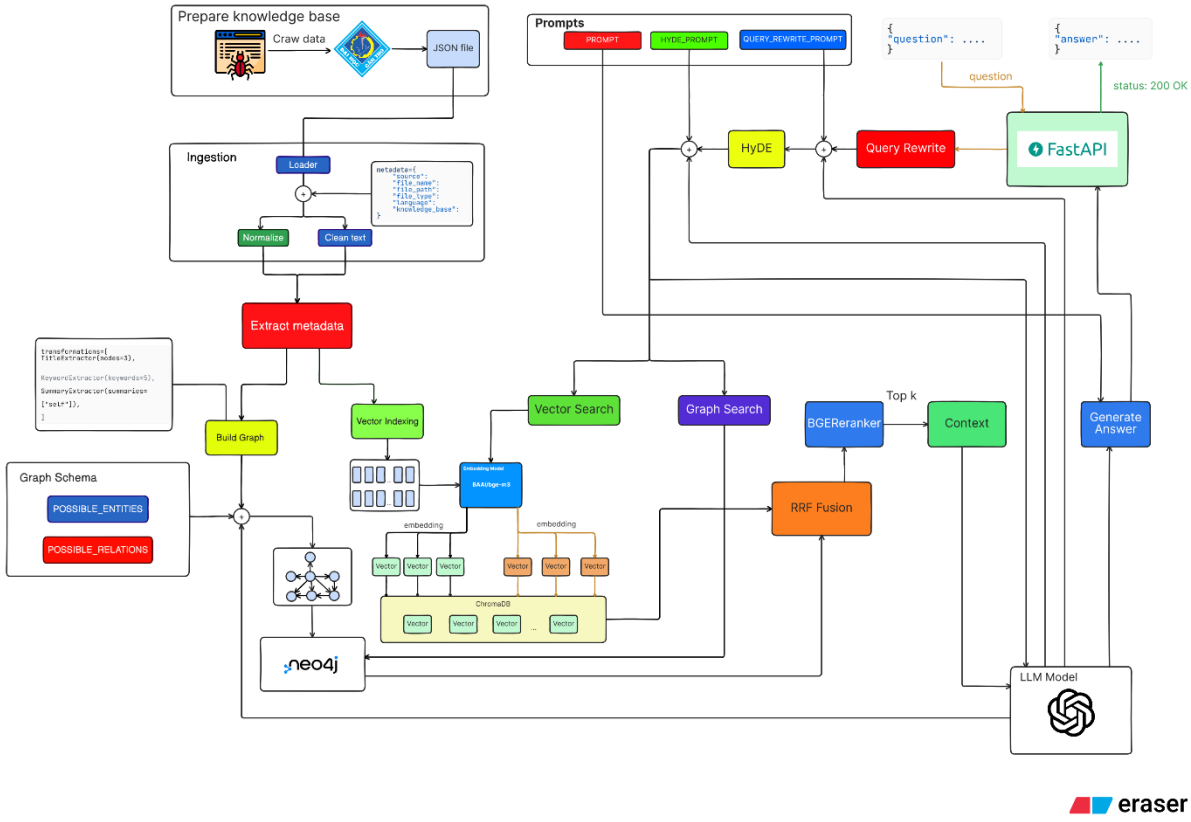


Figure 3.3. Overall Hybrid RAG architecture

During knowledge-base preparation, collected documents are normalized, enriched with metadata, embedded into Chroma, and transformed into entities and relationships in Neo4j. During online consultation, FastAPI rewrites the query and generates HyDE text, retrieves candidates through vector and graph search, combines them using RRF, reranks them, and supplies the selected context to the language model to generate a grounded answer.

3.2.2. Data Design

3.2.2.1. Dataset Structure

The training rows contain MaToHop, DiemToHop, NhomTinhCach, and NhomNganh. During dataset generation, a Holland code is selected stochastically from the compatible types and is considered before the score-based filtering step. When the same combination of subject group, score, and Holland code is compatible with several major groups, one row is emitted for

every valid group. Consequently, identical predictor values may legitimately occur with different target labels.

This construction preserves all eligible recommendations, but it converts an inherently one-to-many recommendation problem into a single-label multiclass training table. A classifier is therefore sometimes asked to choose one class even though several classes are valid. These choices are important when interpreting cross-validation results.

No.	Field name	Role	Data type	Description
1	MaToHop	Input feature	String	Code of the examination subject combination, such as A00, A01, D01, or X01. The dataset contains 27 different combination codes.
2	DiemToHop	Input feature	Float	Total score calculated from the subjects belonging to the corresponding combination. The observed values in the dataset range from 15.0 to 30.0.
3	NhomTinhCach	Input feature	String	Holland personality code. Its possible values are R, I, A, S, E, and C.
4	NhomNganh	Target label	String	Major-group label predicted by the classification model. The dataset contains 19 different major-group classes.

Table 3.10. Structure of the classification dataset

The 2025 examination-score workbook was derived from a publicly released THPT examination dataset[26]. The 2026 workbook was derived from the Tuổi Trẻ dataset[27][28] distributed through the 2026 collection repository. Subject-combination mappings were cross-checked against published combination references and Can Tho University's official admission list[30]. The final classification CSV files were generated programmatically from these score workbooks and the project's major-group and Holland mapping tables; AI was not used to fabricate examination scores.

Dataset	Rows	Classes	Minimum	Median	Maximum	Exact duplicate rows
2025	123,323	19	385	10,000	10,000	93.35%
2026	125,503	21	6	10,000	10,000	93.62%
Mixed	152,506	21	391	10,000	10,000	94.50%

Table 3.11. Characteristics of the generated classification datasets

The duplicate percentage in Table 3.11 counts identical four-column rows, including the label. A high value is plausible because the predictor space is discrete: subject combinations and Holland codes are finite, scores are repeated or rounded, and one source record may be expanded into several eligible labels. It should not be interpreted as proof of data corruption. However, it means that random row-level folds can contain highly similar patterns and may produce optimistic estimates of generalization.

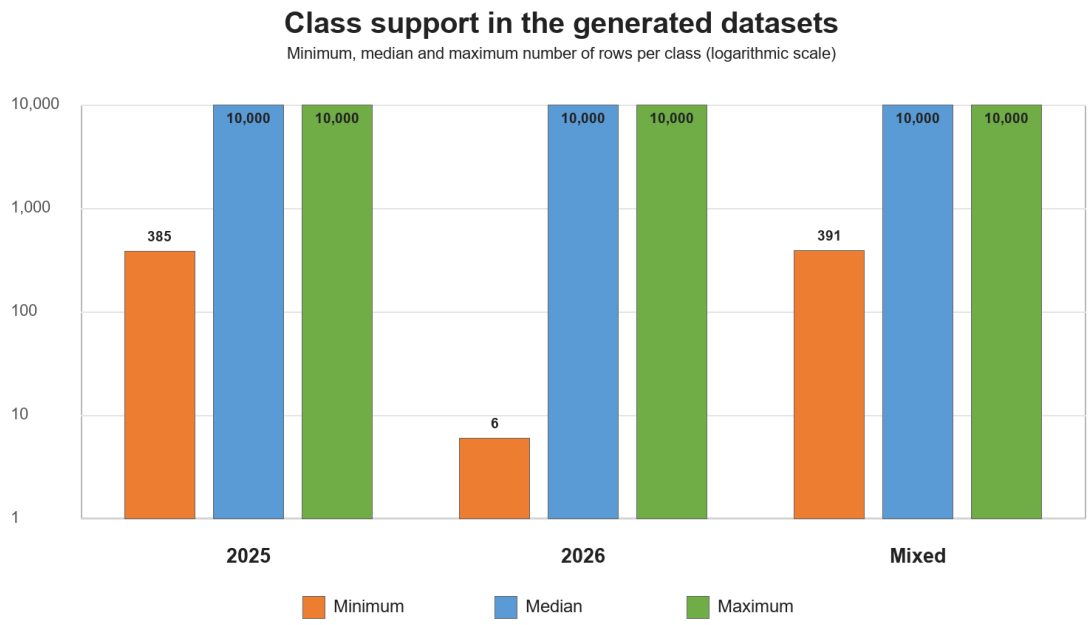


Figure 3.4. Per-class support in the generated datasets (logarithmic scale)

Figure 3.4 shows that the 2026 dataset has the highest aggregate benchmark scores but also contains extremely small classes: the minimum class has only 6 rows, while the 2025 minimum is 385. Therefore, the 2026 weighted scores mainly describe performance on well-supported classes and should not be interpreted as equally strong performance for every major group.

3.2.2.2. Vector Database Structure

Official major documents are divided into manageable chunks. Each chunk is stored with its source metadata and a BGE-M3 embedding in a Chroma collection. At query time, the rewritten query and HyDE text are embedded in the same vector space; cosine-similarity search returns the most semantically relevant chunks for later fusion and reranking.

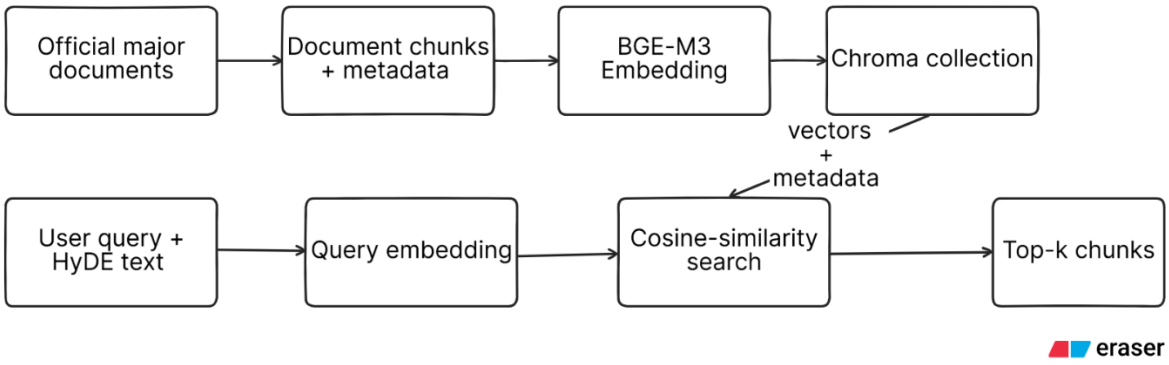


Figure 3.5. Vector database structure and retrieval process

3.2.2.3. Knowledge Graph Schema

Element	Meaning / relationship
MAJOR	A university major represented in the curated knowledge base
MAJOR_GROUP	A broader prediction and consultation category
HOLLAND_TYPE	One vocational-interest type in the RIASEC model
SKILL, SUBJECT, CAREER	Skills, relevant school subjects, and career outcomes associated with a major
BELONGS_TO	Connects a specific major to its major group
MATCHES_HOLLAND	Represents an explicit Holland compatibility relation
REQUIRES_SKILL	Represents skills commonly needed by a major
USES_SUBJECT	Represents relevant school subjects
LEADS_TO_CAREER	Represents possible career outcomes

Table 3.12. Knowledge graph elements and relationships

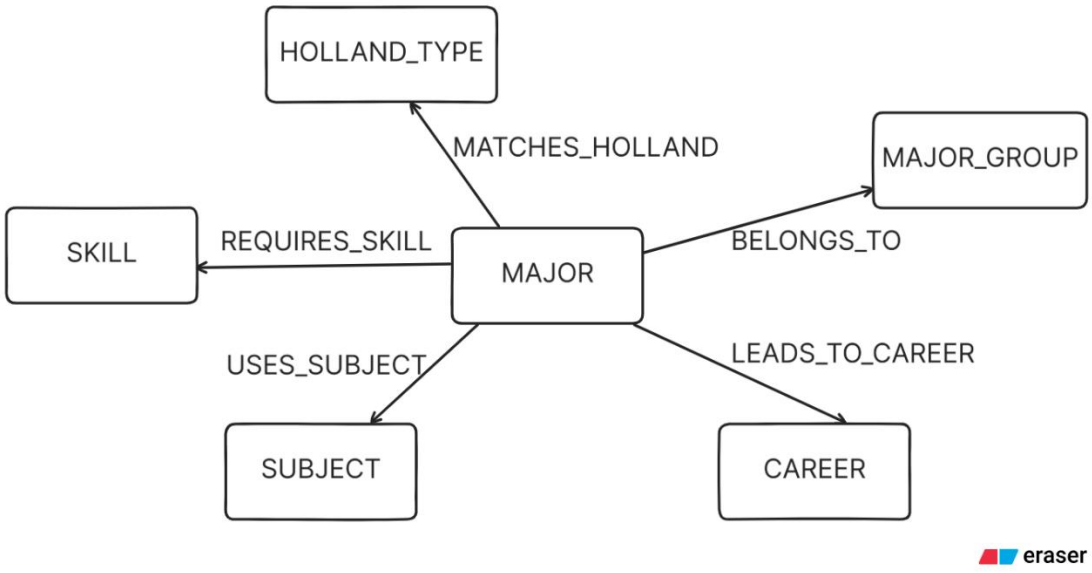


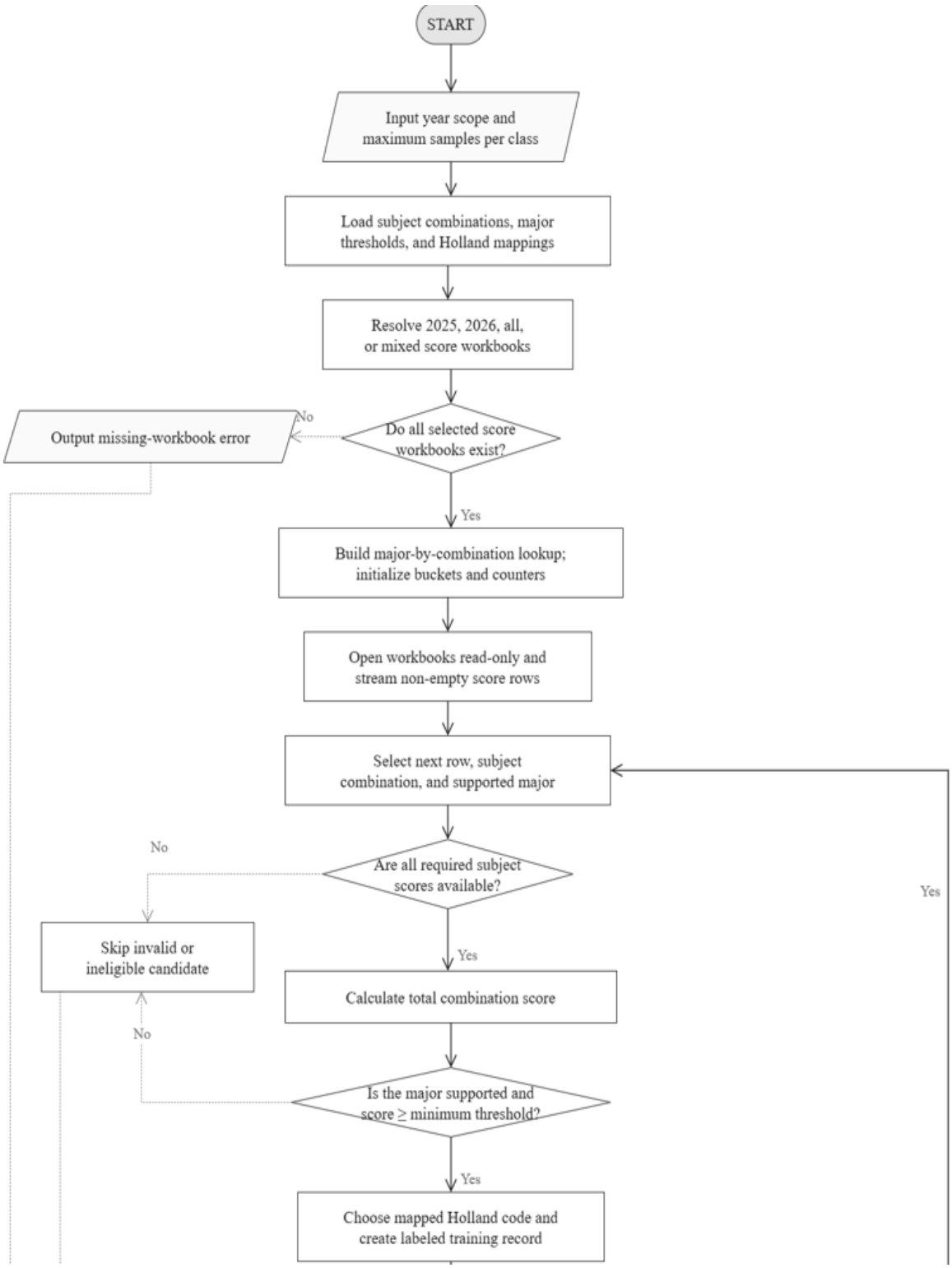
Figure 3.6. Knowledge graph schema for major consultation

No account table or persistent personal-profile database is required in the current prototype. This limits unnecessary personal-data storage and keeps the study focused on recommendation and consultation.

3.2.3. Detailed Design

3.2.3.1. Dataset Generation Algorithm

Figure 3.7 describes how the classification datasets are generated from examination-score workbooks and the project's reference mappings. The algorithm validates the selected sources, streams score rows, constructs supported subject-combination and major candidates, applies score thresholds, and controls class size through reservoir sampling.


START
Input year scope and maximum samples per class
Load subject combinations, major thresholds, and Holland mappings
Resolve 2025, 2026, all, or mixed score workbooks
Do all selected score workbooks exist?
No
Output missing-workbook error
Yes
Build major-by-combination lookup; initialize buckets and counters
Open workbooks read-only and stream non-empty score rows
Select next row, subject combination, and supported major
Are all required subject scores available?
No
Skip invalid or ineligible candidate
Yes
Calculate total combination score
Is the major supported and score ≥ minimum threshold?
No
Yes
Choose mapped Holland code and create labeled training record
Yes

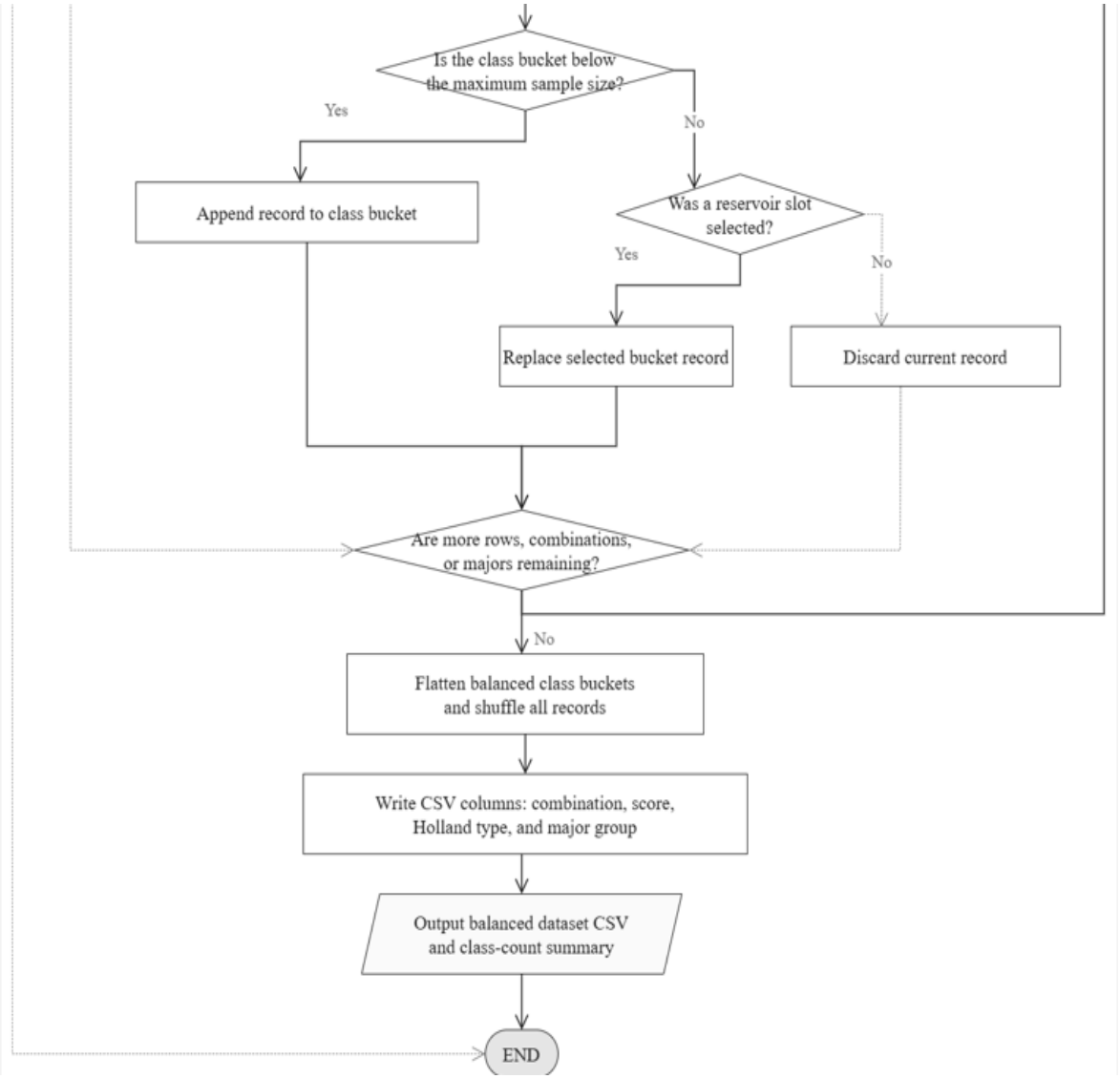


Figure 3.7. Dataset generation algorithm

For each eligible candidate, the algorithm creates a record containing the subject-combination code, total score, compatible Holland type, and major-group label. Records are collected in per-class reservoirs, flattened, shuffled, and exported as a balanced CSV. The repeated filtering and sampling branches explain why the output may contain repeated discrete predictor patterns without implying that examination scores were synthetically generated.

3.2.3.2. Model Training Algorithm

Figure 3.8 summarizes the training implementation. The same pipeline handles numerical and categorical features, performs feature selection, and evaluates hyperparameter

candidates with five-fold cross-validation before saving the selected estimator and its cross-validation results.

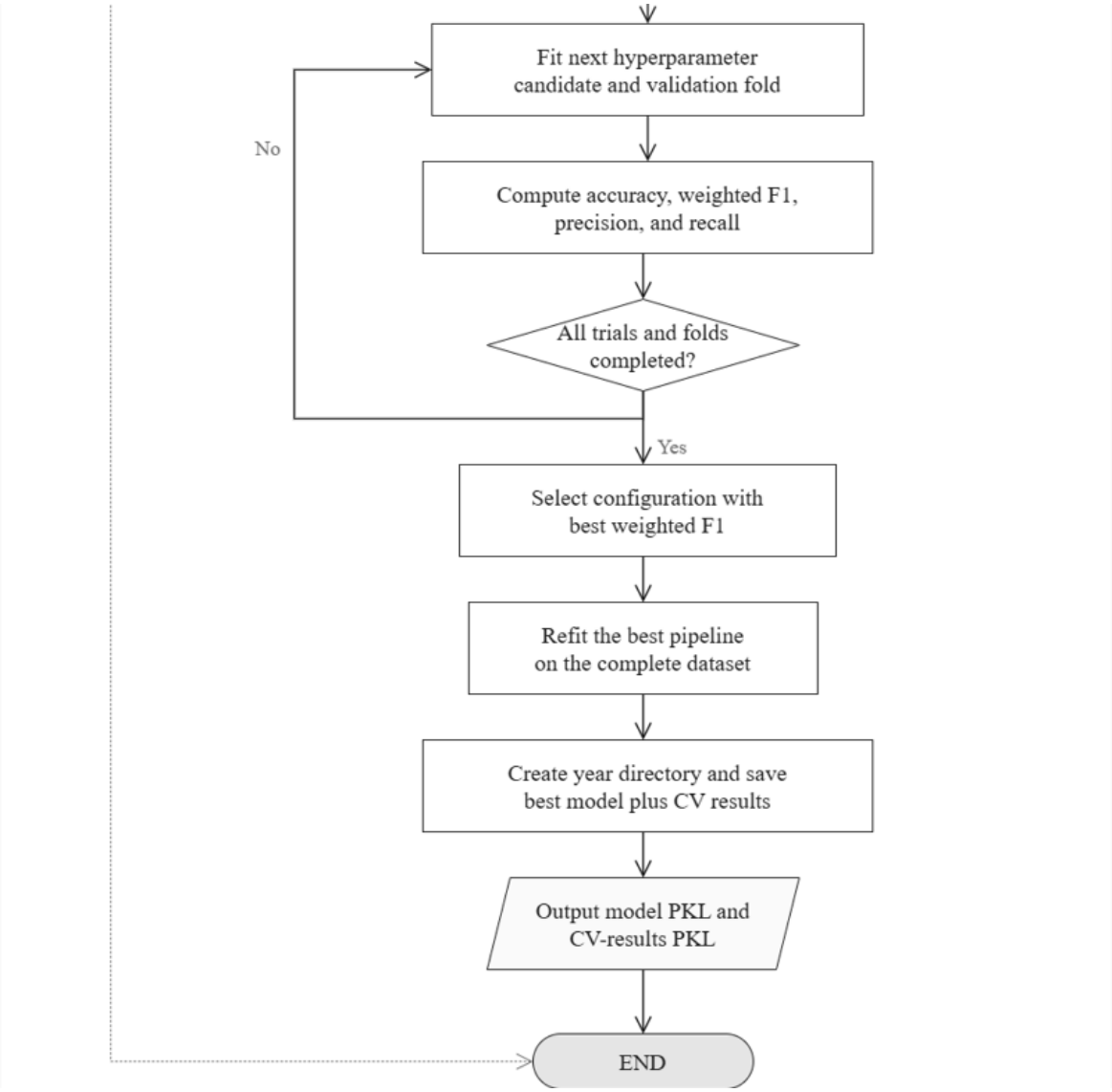


Figure 3.8. Model training algorithm

The numerical score is mean-imputed and standardized, while the subject-combination and Holland fields are mode-imputed and one-hot encoded. SelectKBest reduces the transformed feature set. RandomizedSearchCV then evaluates 30 parameter configurations over five folds and selects the configuration with the highest weighted F1-score; the best pipeline is refitted on the complete dataset and serialized for deployment.

3.2.3.3. Classification Model Evaluation

Four classification algorithms were compared on the 2025, 2026, and mixed-year datasets. The purpose of this experiment was not only to identify a high-scoring model, but also to understand how model assumptions, label construction, class imbalance, and temporal data differences affect the reliability of the recommendations.

Each algorithm used the same preprocessing pipeline: missing-value handling, standardization of the numerical feature, one-hot encoding of categorical features, feature selection, and classification. The benchmark used five-fold stratified cross-validation. Accuracy, weighted precision, weighted recall, and weighted F1-score were recorded as the mean and standard deviation across folds. In a single-label multiclass problem, weighted recall is mathematically equal to accuracy; therefore, the following table reports accuracy, weighted precision, and weighted F1-score to avoid presenting redundant columns.

Dataset	Algorithm	Accuracy	F1-score	Precision
2025	Gradient Boosting	0.627482321	0.607211531	0.649763989
2025	Random Forest	0.624068529	0.606656783	0.634216872
2025	Support Vector Machine	0.618814024	0.582429661	0.628268234
2025	Logistic Regression	0.591365754	0.558255901	0.573782232
2026	Gradient Boosting	0.667099586	0.66067473	0.699407251
2026	Random Forest	0.663593697	0.656914319	0.690469099
2026	Support Vector Machine	0.653450525	0.640517668	0.677004064
2026	Logistic Regression	0.637187981	0.619223021	0.652700667
Mixed	Gradient Boosting	0.604959785	0.581537013	0.634789147
Mixed	Random Forest	0.602336942	0.584575511	0.615358561
Mixed	Support Vector Machine	0.591196411	0.549946004	0.616102922
Mixed	Logistic Regression	0.570016912	0.539895924	0.566831174

Table 3.13. Five-fold cross-validation benchmark results

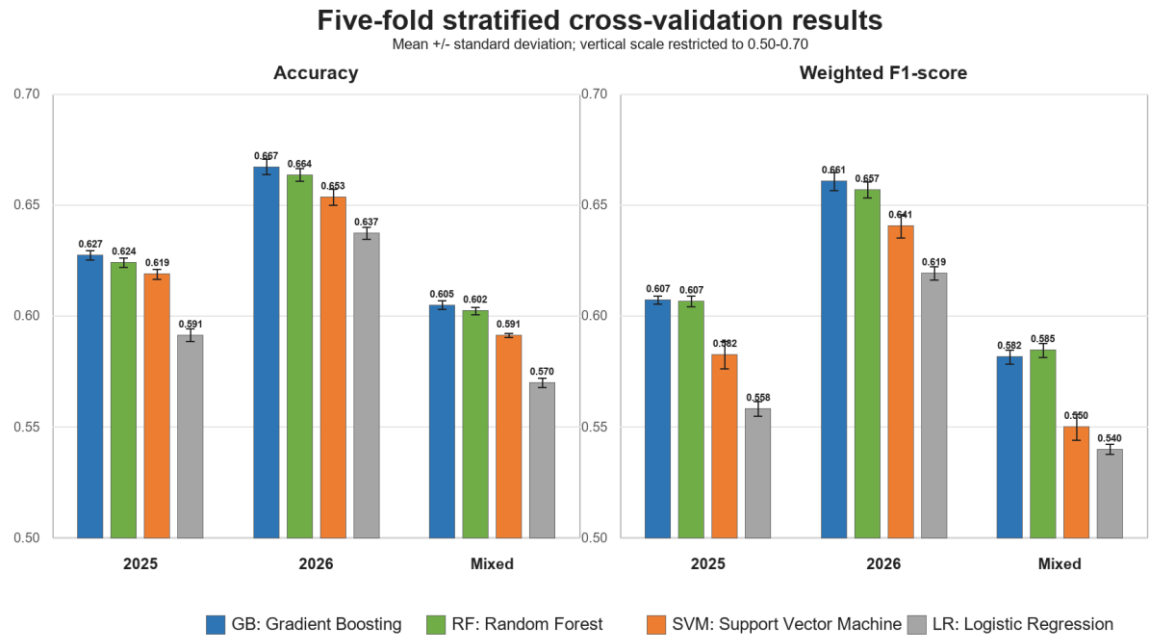


Figure 3.9. Accuracy and weighted F1-score of the four classification algorithms

The differences among the leading tree-based models are small. On the 2025 dataset, Gradient Boosting exceeds Random Forest by only 0.0006 in weighted F1-score. On the 2026 dataset, the difference is 0.0038. On the mixed dataset, Random Forest has a weighted F1-score 0.0030 higher than Gradient Boosting, although Gradient Boosting has slightly higher accuracy and weighted precision. These margins support cautious model selection rather than a claim that one algorithm is universally superior.

Algorithm	Main difference	Strengths in this project	Weaknesses and observed behavior
Logistic Regression	Linear decision functions provide the baseline.	Fast, comparatively easy to interpret, and useful for checking whether complex models add value.	Cannot naturally capture all nonlinear interactions among score, subject group, and Holland type; it produced the lowest weighted F1-score in all three datasets.
Support Vector Machine	Finds maximum-margin class boundaries; nonlinear relations	Competitive on structured data and less restricted than a linear baseline.	Training and probability estimation are more costly on large multiclass data; minority classes and overlapping

Algorithm	Main difference	Strengths in this project	Weaknesses and observed behavior
	can be represented through a kernel.		labels remain difficult. Its results were between the linear and tree ensembles.
Random Forest	Aggregates many independently randomized decision trees.	Models nonlinear interactions, reduces dependence on a single tree, and was the best model by weighted F1-score on the mixed dataset.	Less transparent than a linear model; majority classes can dominate weighted results, and class probabilities may require calibration before being interpreted as confidence.
Gradient Boosting	Builds trees sequentially so that later trees correct residual errors.	Captured feature interactions effectively and achieved the highest weighted F1-score on the 2025 and 2026 datasets.	More sensitive to hyperparameters and noisy or contradictory labels; sequential fitting is slower and may concentrate on ambiguous examples.

Table 3.14. Differences, strengths, and limitations of the compared algorithms

The selection criterion was weighted F1-score because the classes were imbalanced and both missed and incorrect recommendations were relevant. Gradient Boosting was selected for the 2025 and 2026 datasets, while Random Forest was selected for the mixed dataset. RandomizedSearchCV evaluated 30 parameter combinations with five-fold cross-validation and refitted the selected configuration using weighted F1-score.

Dataset	Selected algorithm	Accuracy	Weighted precision	Weighted F1-score	MSE	MAE
2025	Gradient Boosting	0.6280 +/- 0.0015	0.6512 +/- 0.0038	0.6076 +/- 0.0019	0.023401	0.046994
2026	Gradient Boosting	0.6678 +/- 0.0013	0.6988 +/- 0.0013	0.6622 +/- 0.0014	0.020103	0.041655

Dataset	Selected algorithm	Accuracy	Weighted precision	Weighted F1-score	MSE	MAE
Mixed	Random Forest	0.6020 +/- 0.0017	0.6166 +/- 0.0034	0.5840 +/- 0.0023	0.022418	0.044864

Table 3.15. Cross-validation results of the selected tuned configurations

The selected 2025 Gradient Boosting configuration used 200 estimators, learning rate 0.05, maximum depth 3, minimum split size 5, minimum leaf size 1, and 10 selected features. The 2026 configuration used 200 estimators, learning rate 0.03, maximum depth 2, minimum split size 2, minimum leaf size 1, and 10 selected features. The mixed Random Forest configuration used 100 trees, unrestricted maximum depth, minimum split size 5, minimum leaf size 2, and 10 selected features.

MSE and MAE in Table 3.15 were calculated between the one-hot encoded true major-group labels and the class-probability vectors returned by each saved pipeline; lower values indicate that the predicted probabilities are closer to the observed labels. The 2026 Gradient Boosting model produced the lowest MSE (0.020103) and MAE (0.041655), followed by the mixed Random Forest model (0.022418 and 0.044864) and the 2025 Gradient Boosting model (0.023401 and 0.046994). These probability-error measures supplement, rather than replace, accuracy and weighted F1-score. They were calculated on the complete datasets using the saved refitted pipelines, so they are in-sample diagnostics and should not be interpreted as independent generalization estimates. Direct cross-year comparison also requires caution because the datasets contain different numbers and distributions of classes.

The tuning and benchmark runs used different cross-validation splitter configurations: the benchmark used shuffled stratified folds with a fixed random seed, whereas RandomizedSearchCV used its default five-fold splitter through cv=5. Accordingly, the small numerical changes between Tables 3.13 and 3.15 should not be presented as a strict before-and-after improvement. In particular, the mixed tuned score is slightly lower than its benchmark score; this is within the variation expected from the different folds.

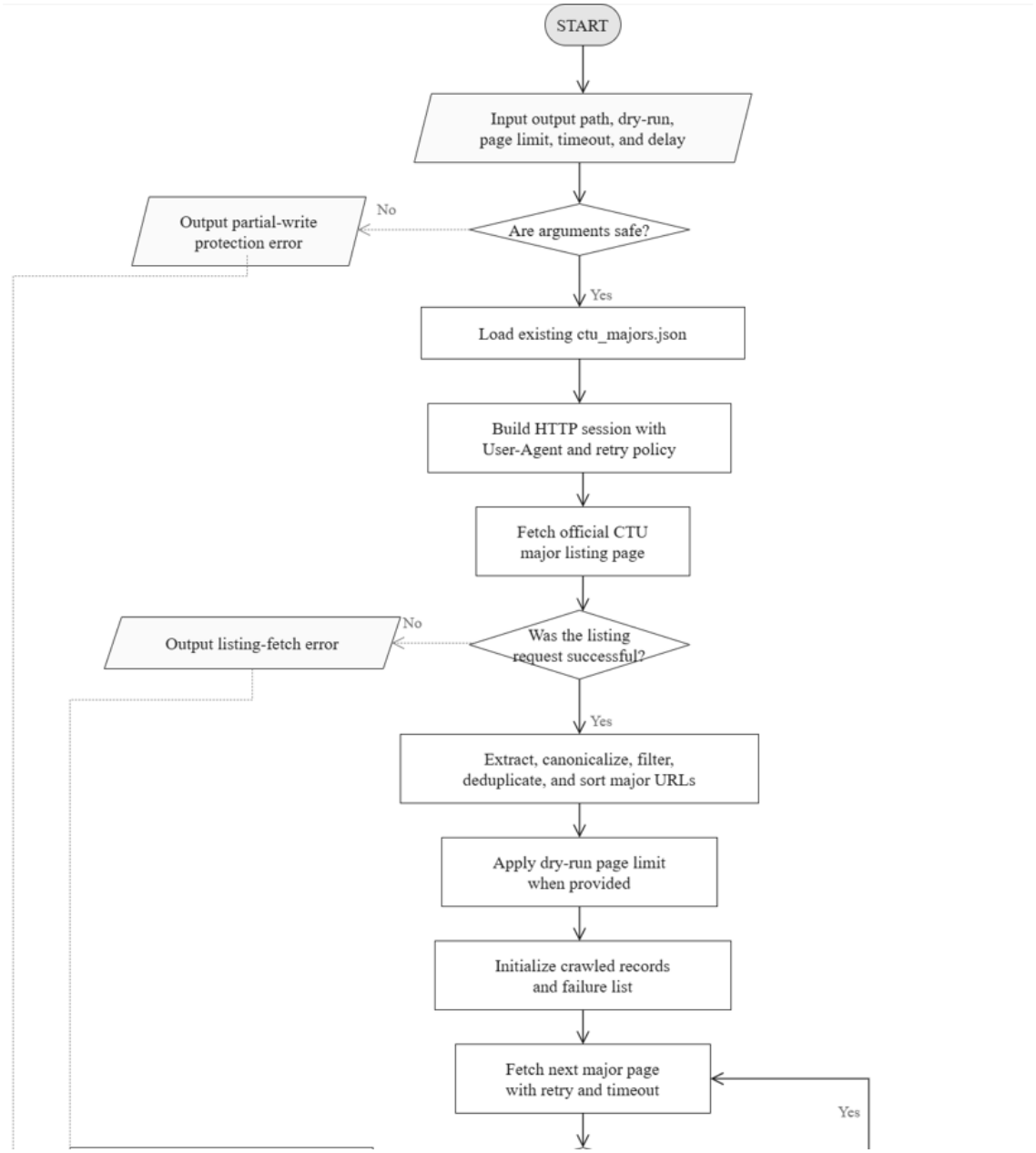
The backend loads the tuned 2025 Gradient Boosting model the default model. This choice does not mean that its aggregate score is the highest among all years; the 2026 model has higher weighted metrics. The 2025 model was selected as the safer release model because every 2025 class has at least 385 rows, whereas the 2026 data contain classes with only 6, 9, 17, and 53 rows. The 2025 data therefore provide more defensible class coverage for the current

demonstration. The model returns ranked Top-3 major-group candidates, after which the application and Hybrid RAG layer provide further evidence and explanation.

The reported values are comparative cross-validation estimates, not proof of real-world counseling accuracy. First, the randomly assigned compatible Holland code is rule-generated rather than obtained from an actual psychometric assessment for every candidate; the model can therefore reproduce the dataset-generation rules more strongly than real human preference. Second, one predictor tuple may have several valid major-group labels, creating unavoidable ambiguity for a single-label classifier. Third, weighted averages are dominated by common classes and can hide failure on rare classes, including cases where a classifier predicts no samples for a rare label. Finally, the year-specific and mixed datasets reflect different admission structures and should not be treated as interchangeable.

3.2.3.4. CTU Major-Information Crawling Algorithm

Figure 3.10 presents the defensive update process used to enrich ctu_majors.json from official Can Tho University pages[31] .It is designed for scheduled weekly execution while preventing a temporary network or parsing failure from corrupting the existing knowledge source.


START
Input output path, dry-run, page limit, timeout, and delay
Are arguments safe?
No
Output partial-write protection error
Yes
Load existing ctu_majors.json
Build HTTP session with User-Agent and retry policy
Fetch official CTU major listing page
Was the listing request successful?
No
Output listing-fetch error
Yes
Extract, canonicalize, filter, deduplicate, and sort major URLs
Apply dry-run page limit when provided
Initialize crawled records and failure list
Fetch next major page with retry and timeout
Yes

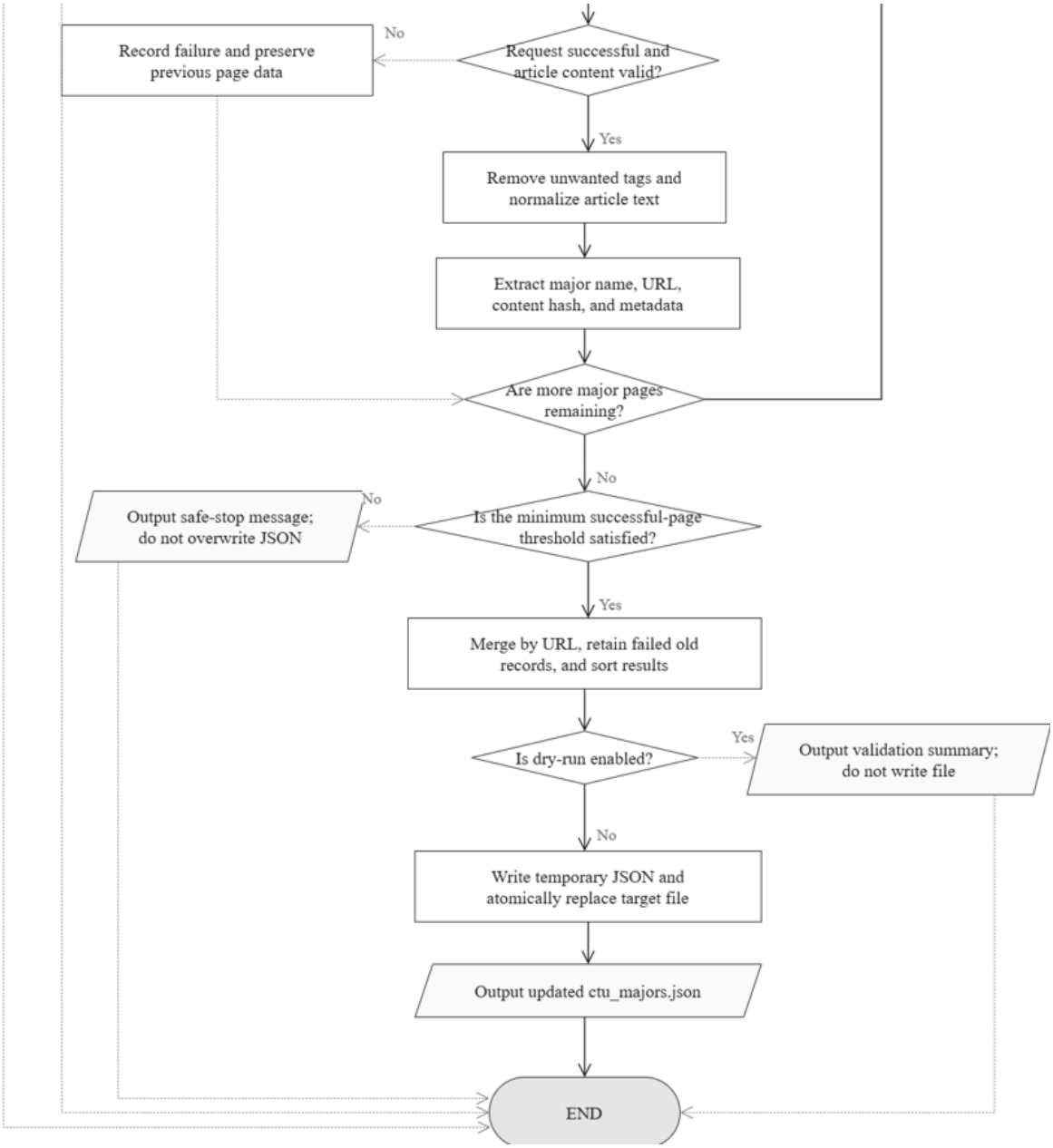


Figure 3.10. CTU major-information crawling and update algorithm

The crawler loads the previous JSON, discovers and canonicalizes official major URLs, retries failed requests, validates article content, and records a content hash and metadata for successful pages. Failed pages retain their previous records. A minimum-success threshold blocks unsafe replacement, dry-run mode reports proposed changes without writing, and a successful run writes a temporary file before atomically replacing the target JSON.

3.2.3.5. Major-Group Prediction Function

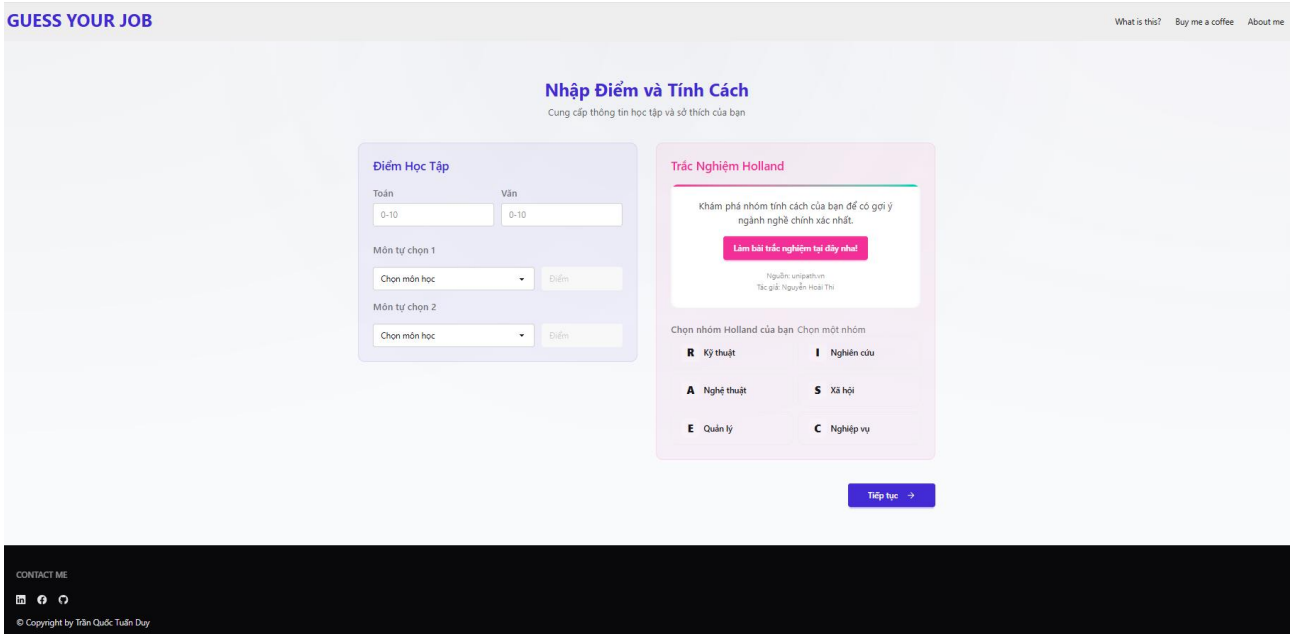


Figure 3.11. Examination-score and Holland-type input interface

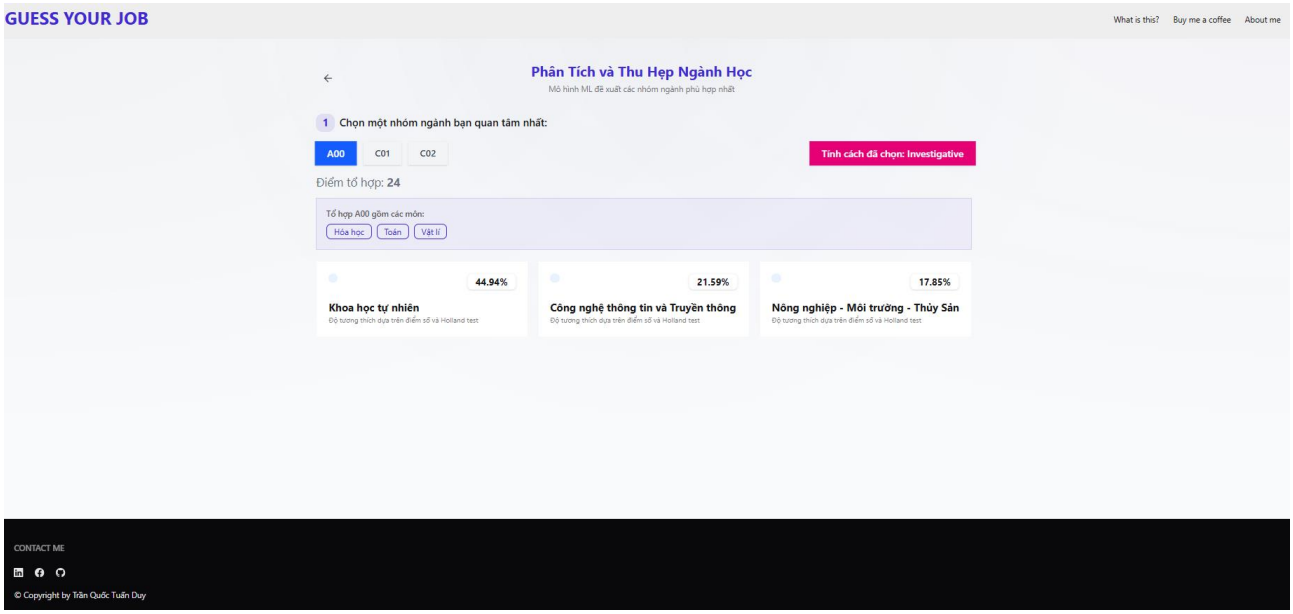
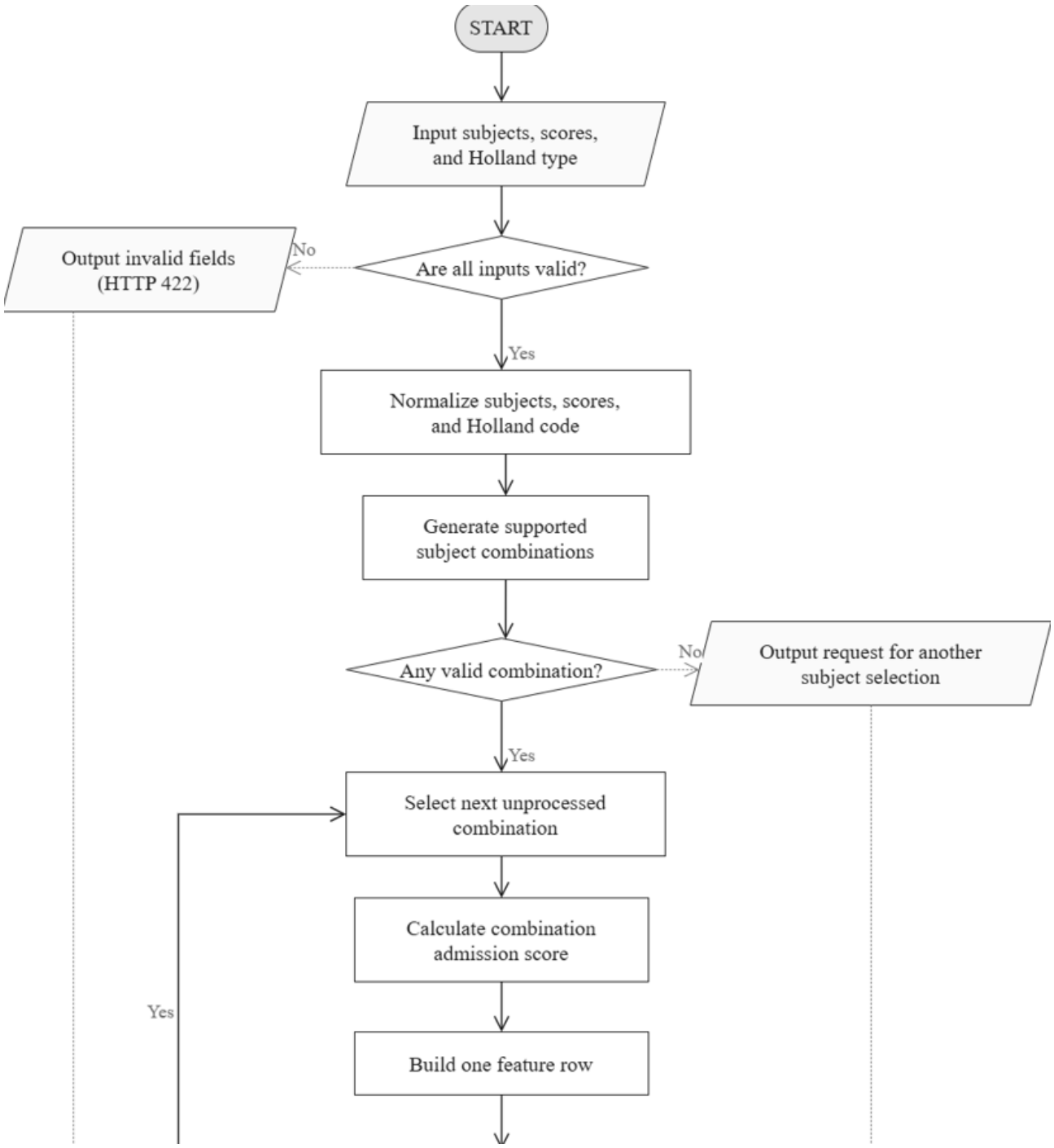


Figure 3.12. Major-group recommendation results interface

Interface control	User action	System response
Subject selectors	Choose two distinct optional subjects	Prevent duplicate selection and update the associated score fields

Interface control	User action	System response
Score inputs	Enter values from 0 to 10	Validate range and required values
Holland selector	Choose R, I, A, S, E, or C	Store the standardized categorical value
Recommend button	Submit valid input	Call /predict and show a loading/error state
Result cards	Review and select a ranked group	Display combinations, scores, Top-3 groups, and probabilities
Subject-combination mapping	Query	Find combinations supported by the selected subjects
Scores	Calculate	Sum the three component-subject scores for each combination
Trained model	Predict	Return a probability distribution over major-group classes
Ranked result	Create	Sort probabilities and retain the three highest values

Table 3.16. Interface controls and data operations for major-group prediction


START
Input subjects, scores, and Holland type
Are all inputs valid?
No
Output invalid fields (HTTP 422)
Yes
Normalize subjects, scores, and Holland code
Generate supported subject combinations
Any valid combination?
No
Output request for another subject selection
Yes
Select next unprocessed combination
Calculate combination admission score
Build one feature row
Yes

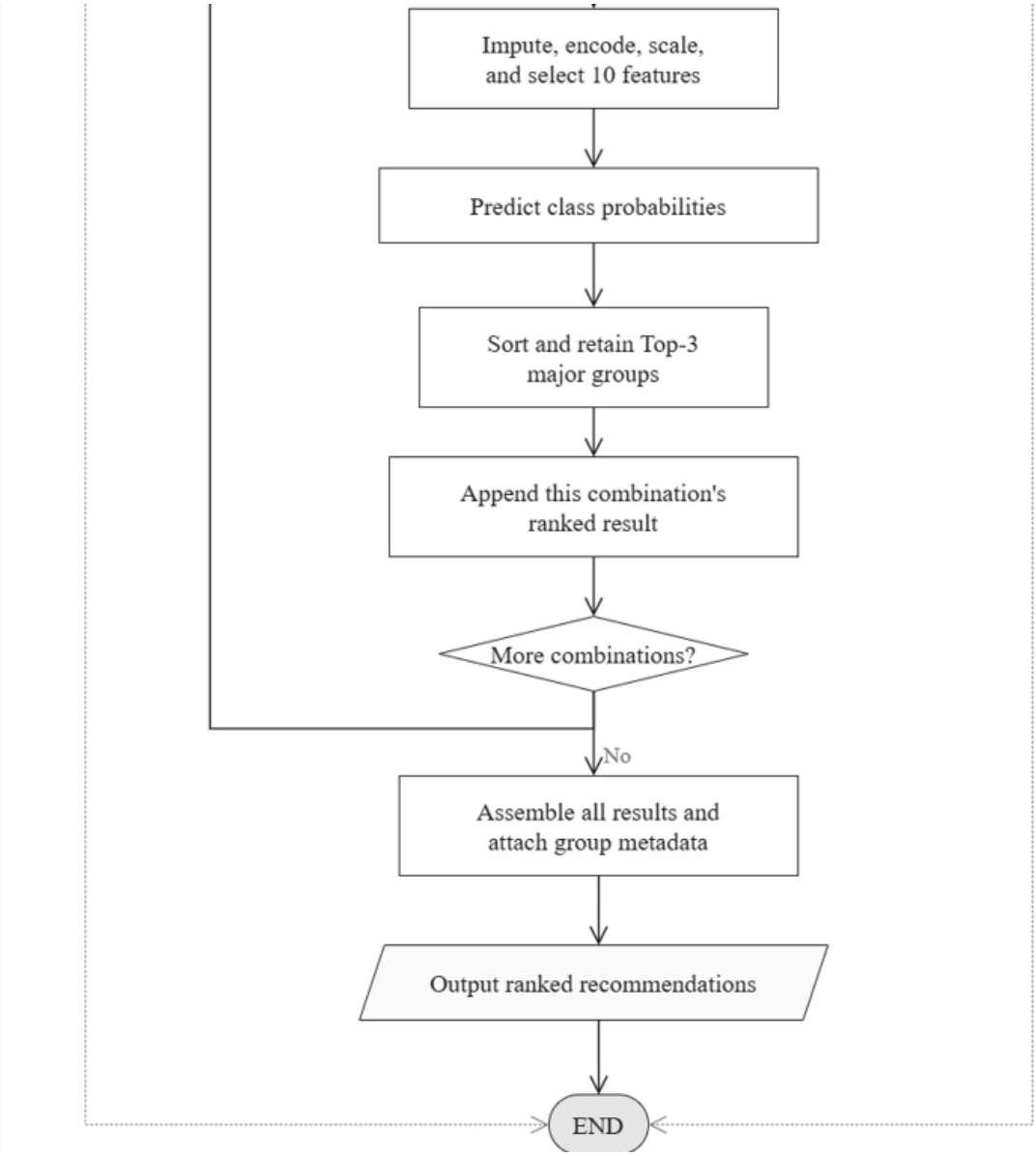


Figure 3.13. Major-group prediction flowchart

The deterministic validation and score-calculation steps occur before classification. Therefore, the model does not infer unsupported combinations or change a student's examination score. The result is a recommendation score for exploration, not an admission guarantee.

3.2.3.6. Hybrid RAG Consultation Function

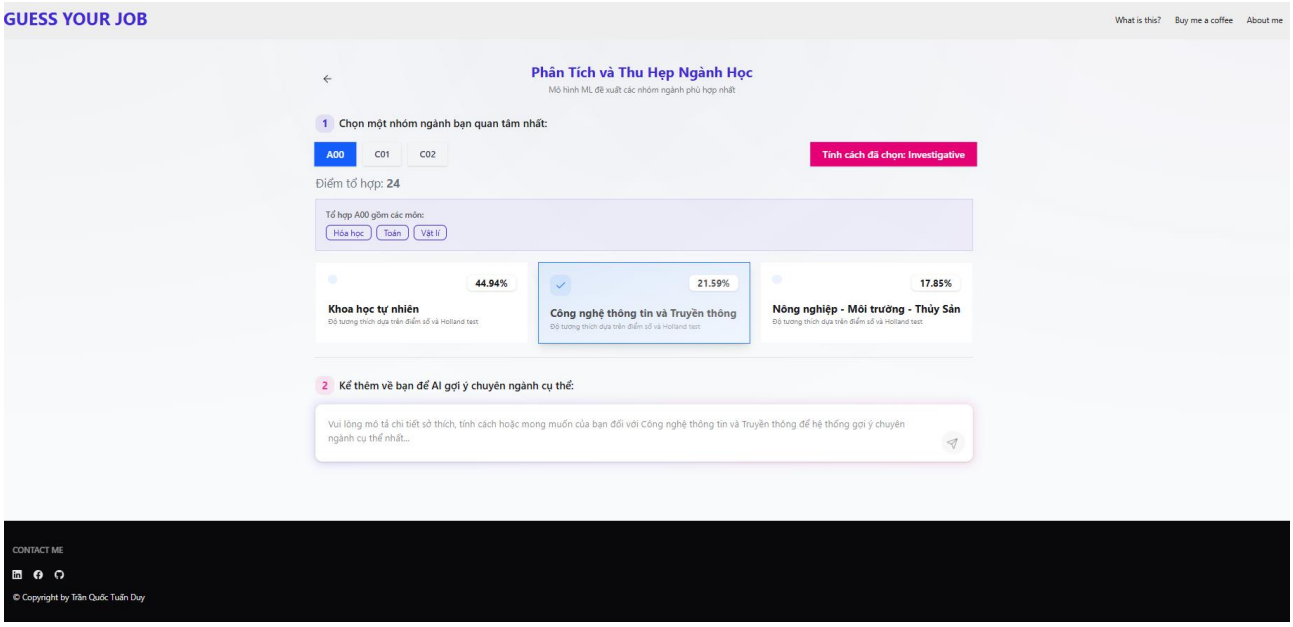


Figure 3.14. Major-group selection and personal-description interface

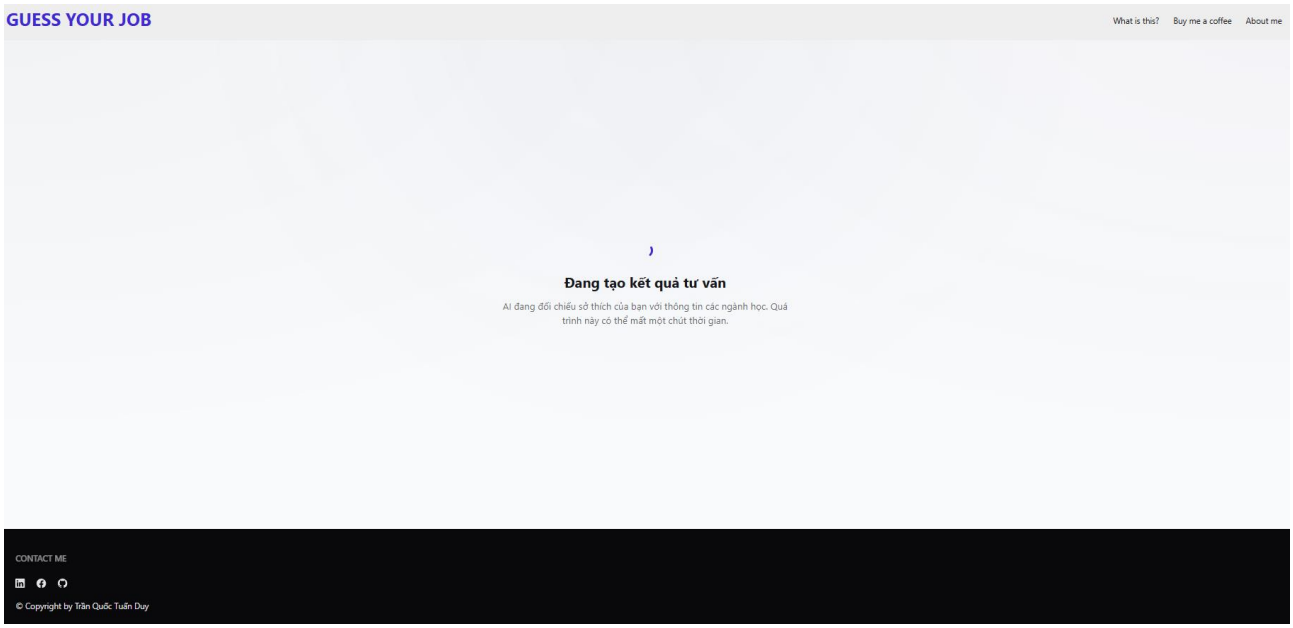


Figure 3.15. Consultation generation loading state

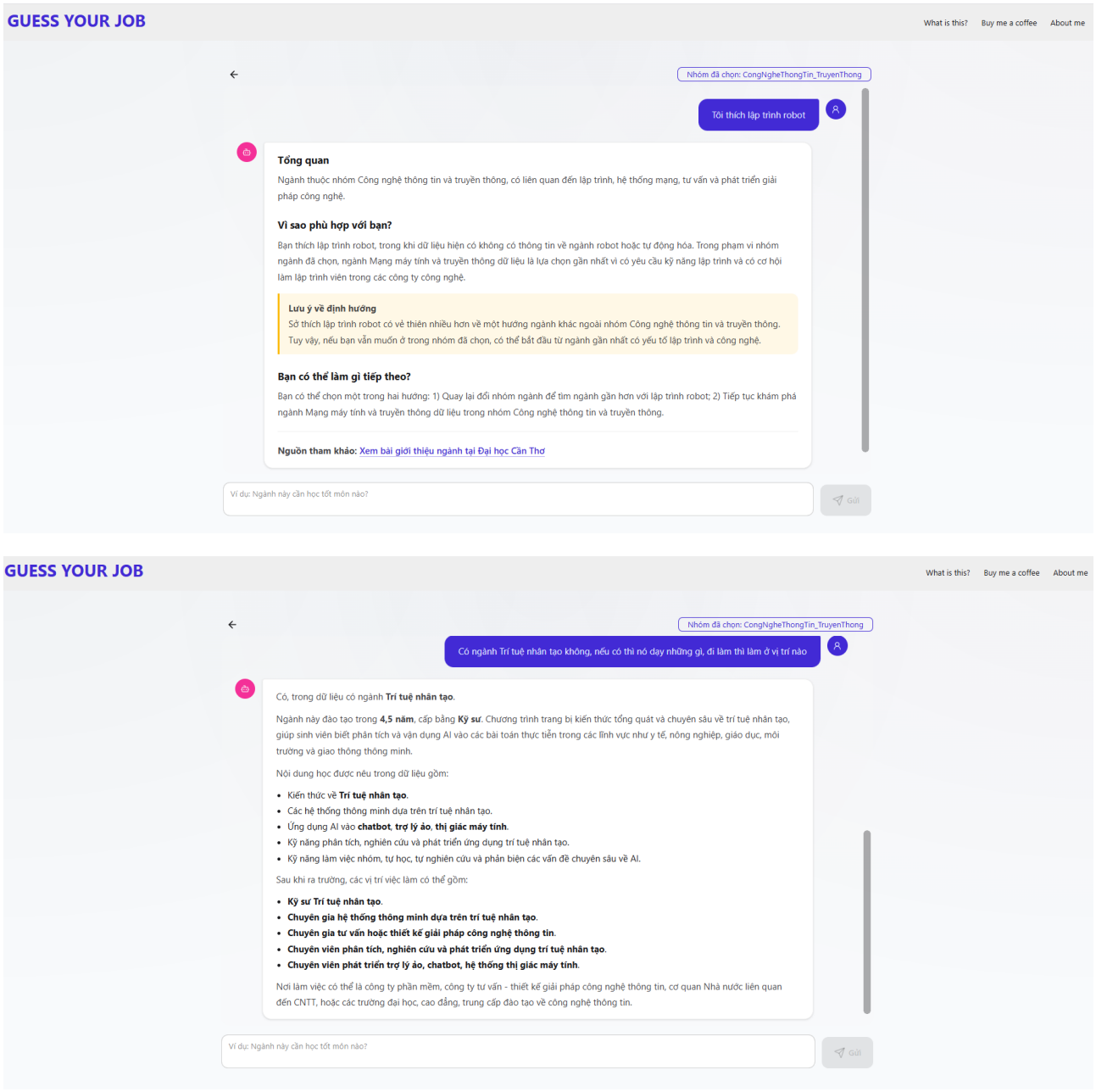


Figure 3.16. Hybrid RAG consultation results and reference-source interface

Interface control	User action	System response
Major-group card	Select one predicted group	Set the consultation scope
Personal-description field	Describe interests, strengths, or goals	Include the description in the consultation request
Consult button	Submit the request	Call /chat and show progress

Interface control	User action	System response
Consultation panel	Read the generated sections	Show description, suitability reasons, orientation, and suggested questions
Chat input	Ask a follow-up question	Send the question with bounded recent history
Reference links	Open a cited source	Navigate to the accepted official source URL

Table 3.17. Interface controls and data operations for Hybrid RAG consultation


START
Input group, question, and recent history
Is the request valid?
No
Output validation error
Yes
Combine question with relevant history
Rewrite standalone query
Usable rewritten query?
No
Use original contextual query
Yes
Select rewritten or original fallback query
Generate hypothetical answer (HyDE)
Usable HyDE text?
No
Use selected query for semantic retrieval
Yes
Vector search in Chroma
Graph search in Neo4j
Retriever result state?
None
Output insufficient-evidence response
One
Use the available list
Both

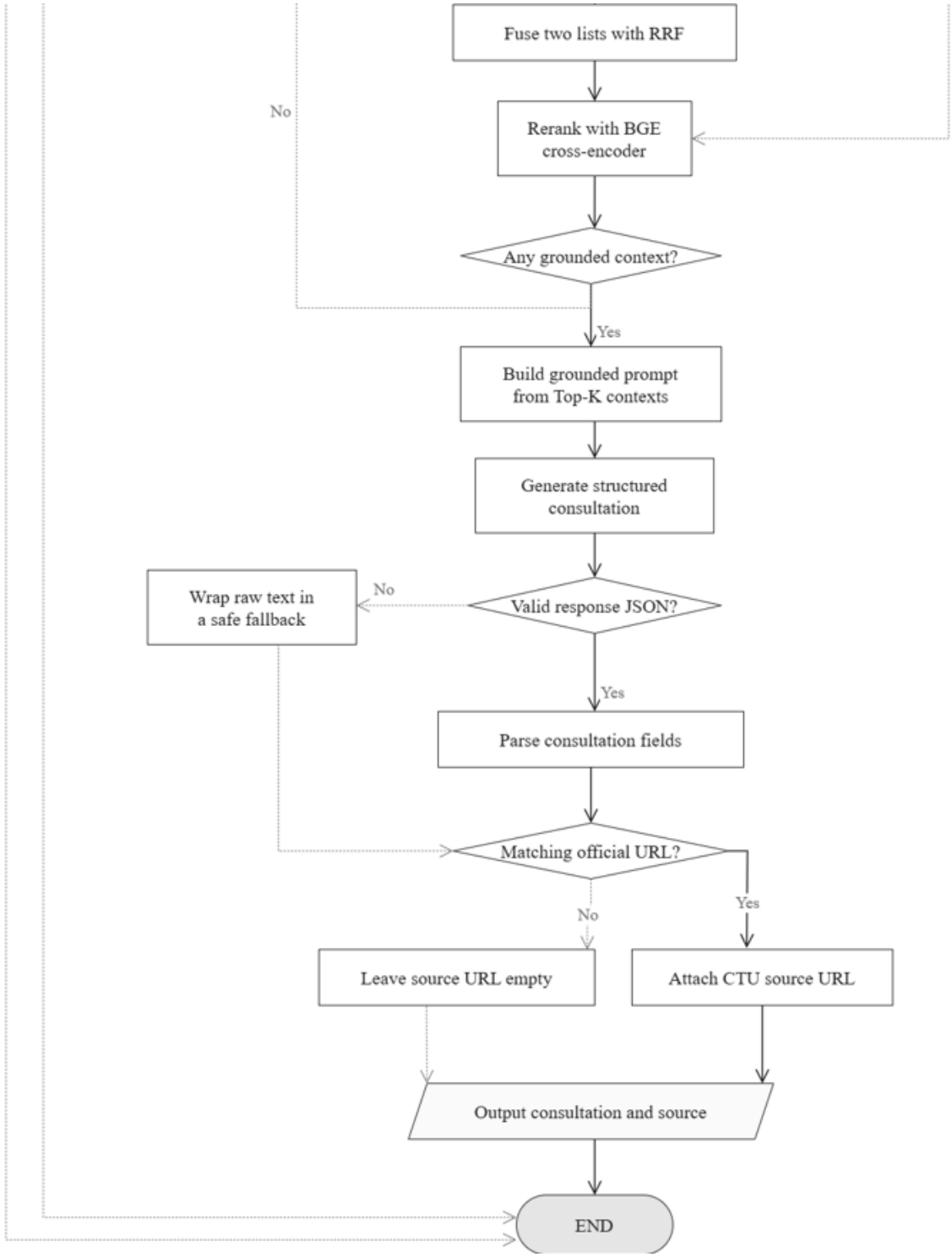


Figure 3.17. Hybrid RAG consultation flowchart

Vector search captures semantic similarity, whereas graph search captures explicit domain relationships that may not be expressed in the same wording. RRF provides a score-independent way to combine their ranked lists, and the reranker evaluates query–candidate relevance before

the final contexts are selected. The final response is constrained to the retrieved evidence and accepted source metadata.

Available: https://guessyourjob.vercel.app/

3.3. Testing

3.3.1. Test Plan

Testing is divided into unit/integration checks for deterministic rules and APIs, experimental evaluation of the four classifiers, a production frontend build, and end-to-end checks for the Hybrid RAG workflow. A test is reported as passed only when execution evidence is available; otherwise its status remains pending.

ID	Test item	Input/ condition	Expected result	Level
TP-01	Input validation	Missing score, score outside [0,10], or duplicated optional subject	Submission is blocked and a specific message is shown	SYS
TP-02	Combination generation	A set of four valid subjects	All and only supported combinations are produced with correct totals	U/I
TP-03	Prediction API	Valid scores and Holland type	Each combination contains exactly three ranked groups and probabilities	INT
TP-04	Model evaluation	Benchmark dataset and fixed evaluation procedure	Accuracy, weighted precision, weighted recall, and weighted F1 are recorded for four algorithms	EXP
TP-05	Top-3 ranking	A valid probability distribution from the loaded model	Exactly three groups are returned in descending probability order for each valid combination	U/I
TP-06	Initial Hybrid RAG consultation	Selected major group and personal description	The system retrieves evidence, produces a grounded consultation, and returns accepted reference metadata	I/S

ID	Test item	Input/ condition	Expected result	Level
TP-07	Follow-up question switching	Ask question 1, switch to a tagged question 2, then return to question 1	Each question thread remains selectable and receives answers in the correct conversation context	S/UI
TP-08	RAG retrieval evaluation	Sixty questions: 58 labeled with expected official CTU URLs and two marked out of scope	The four retrieval configurations are compared using Hit Rate, MRR, Precision, Recall, nDCG, and latency; out-of-scope cases are excluded from retrieval averages	EXP
TP-09	Reference-source filtering	Retrieved metadata containing accepted, malformed, and unaccepted URLs	Only valid URLs from the configured official CTU admission domain are exposed to the user	S/I
TP-10	Insufficient evidence or dependency failure	Out-of-scope query or unavailable Chroma, Neo4j, reranker, or LLM	A controlled limitation or retry message is returned without fabricating a definitive answer	R/S
TP-11	CTU crawler safe update	Dry-run, partial page failures, and a run below the success threshold	Dry-run performs no write; failed old records are retained; unsafe replacement is blocked	INT
TP-12	Weekly knowledge refresh	Scheduled or manual weekly crawler execution	Official pages are validated and ctu_majors.json is replaced only after a successful crawl	OPS
TP-13	Frontend production build	Run the configured Vite production build	The build completes and optimization warnings are recorded without being reported as failures	BLD

ID	Test item	Input/ condition	Expected result	Level
TP-14	Prediction performance	Representative /predict requests after model loading	At least 95% of requests complete within 3 seconds under the demonstration workload	PERF
TP-15	Consultation performance	Representative /chat requests with dependencies available	The loading state appears immediately and at least 95% of requests complete within 40 seconds	PERF
TP-16	User acceptance	First-time students complete the three-step workflow	Users finish without technical assistance and understand fields, ranks, errors, and sources	UAT
TP-17	Responsive and accessible interaction	Desktop/mobile viewports and keyboard navigation	Core controls remain visible, labelled, reachable, and usable at the target viewports	S/U

Table 3.18. System test plan

Level codes: U = unit, I/INT = integration, S/SYS = system, EXP = experimental, R = reliability, OPS = operations, BLD = build, PERF = performance, UI = interface testing, and UAT = user acceptance testing; a slash denotes combined levels.

Unit and integration testing cover deterministic score rules, API contracts, ranking, retrieval, source filtering, and crawler safety paths using controlled inputs and direct execution of the corresponding Python modules. Experimental evaluation uses scikit-learn five-fold cross-validation and RandomizedSearchCV for classification, while the RAG evaluator uses LlamaIndex retrieval metrics over a labeled 60-question set containing 58 answerable and two out-of-scope cases. System testing covers the React/Vite production build, question-thread switching, dependency failures, and the scheduled knowledge refresh. Performance, accessibility, and user-acceptance cases remain pending until their stated measurement procedures are executed.

3.3.2. Results and Analysis

3.3.2.1. Executed System Checks

ID	Observed evidence	Status
TP-02	For Mathematics 8.5, Literature 7.5, Physics 8.0, and English 8.5, the backend generated A01 = 25.0, C01 = 24.0, and D01 = 24.5.	Passed
TP-03	For A01 = 25.0 and Holland I, the loaded model returned SuPhamTinHoc = 0.7902, KhoaHocTuNhien = 0.1038, and KyThuat_CongNghe = 0.0474 in descending order.	Passed
TP-04	Five-fold results were recorded for four algorithms on the 2025, 2026, and mixed datasets; selected configurations were tuned and compared in Tables 3.13-3.15.	Passed
TP-08	The retrieval-only run completed all 60 cases for vector, graph, hybrid, and full configurations. On the 58 answerable cases, the full pipeline achieved Hit Rate 1.0000 and Recall 0.9914; vector achieved the highest MRR 0.8670 and nDCG 0.8971.	Passed
TP-13	The Vite production build completed successfully; the reported bundle-size warning was retained as an optimization item.	Passed with warning

Table 3.19. Executed system-check results

The executed checks confirm that the deterministic combination calculation, model inference path, ranking order, and frontend compilation work with the inspected project. The Vite warning does not prevent deployment, but code splitting or lazy loading should be considered to reduce the initial JavaScript payload. The pending cases must be completed before claiming that the Hybrid RAG and numerical nonfunctional targets are satisfied.

3.3.2.2. Hybrid RAG Retrieval Evaluation

The evaluation module was implemented with LlamaIndex Evaluation [34]. The recorded retrieval-only run evaluated vector, graph, hybrid, and full retrieval configurations at Top-5 over 60 Vietnamese questions. Fifty-eight answerable cases were labeled with expected official CTU URLs, while two out-of-scope cases were retained to test controlled behavior but excluded from the retrieval-metric averages. This design measures source retrieval independently of answer generation.

Metric	Vector	Graph	Hybrid	Full
Test cases	60	60	60	60
Answerable cases	58	58	58	58
Hit Rate	0.9828	0.7069	0.9655	1.0000
MRR	0.8670	0.6485	0.8003	0.8635
Precision@5	0.2928	0.3906	0.2911	0.3247
Recall@5	0.9828	0.6810	0.9483	0.9914
nDCG@5	0.8971	0.6426	0.8286	0.8905
Mean time (s)	0.3472	4.0463	4.3718	36.2974

Table 3.20. Hybrid RAG retrieval evaluation summary

The full configuration achieved complete coverage with Hit Rate 1.0000 and the highest Recall@5 of 0.9914. Its Precision@5 of 0.3247 was also higher than vector and hybrid retrieval, indicating that query rewriting, HyDE, fusion, and reranking reduced some irrelevant results. However, vector retrieval obtained the highest MRR (0.8670) and nDCG@5 (0.8971), narrowly exceeding the full pipeline (MRR 0.8635; nDCG@5 0.8905). Therefore, the additional stages improved coverage but did not uniformly improve the rank of the first relevant source.

Graph retrieval produced the highest Precision@5 (0.3906) but substantially lower Hit Rate (0.7069), Recall@5 (0.6810), and nDCG@5 (0.6426), showing that its smaller result set was more selective but missed relevant sources more often. Hybrid retrieval improved coverage over graph alone, yet remained below the vector baseline on Hit Rate, MRR, Recall, and nDCG. The full pipeline required a mean 36.2974 seconds per case, compared with 0.3472 seconds for vector retrieval, because it includes LLM-based query rewriting and HyDE plus graph retrieval and reranking. The results support using the full configuration when maximum source coverage is the priority, while vector retrieval remains the strongest efficiency baseline. Because this run was retrieval-only, it does not establish answer faithfulness, relevance, or correctness.

CHAPTER 4: CONCLUSION

4.1. Project Summary

This project developed a web-based major recommendation and consultation system that combines Vietnamese high-school examination scores with the Holland RIASEC vocational-interest model. The system validates the user's subject information, derives supported admission combinations, calculates combination scores, and uses a trained multiclass classifier to rank suitable major groups. It then allows the user to select a recommended group and request a more detailed explanation.

The machine learning stage established a common preprocessing pipeline and compared Logistic Regression, Support Vector Machine, Random Forest, and Gradient Boosting using five-fold stratified cross-validation. Gradient Boosting was selected for the 2025 and 2026 datasets, while Random Forest achieved the highest weighted F1-score on the mixed dataset. The tuned 2025 Gradient Boosting model was deployed because the 2025 data provided more defensible support across classes for the current demonstration. The resulting probabilities are used to rank Top-3 candidates rather than to assert a guaranteed admission outcome.

The consultation stage implemented a Hybrid RAG architecture. Official major information is represented as document embeddings in Chroma and as entities and relationships in Neo4j. Query rewriting, HyDE, vector search, graph search, Reciprocal Rank Fusion, BGE reranking, and a configurable large language model are combined to generate answers grounded in retrieved evidence. React and Vite provide the client interface, while FastAPI integrates the prediction and consultation services.

Overall, the main project objectives were achieved at prototype level: the study established the theoretical foundation, constructed and evaluated classification datasets, implemented the recommendation workflow, developed Hybrid RAG consultation, connected the frontend and backend, and recorded the principal test and model-comparison results. The system demonstrates how predictive and generative methods can complement each other in career-orientation support.

4.2. Limitations

The current results must be interpreted cautiously. The Holland code in the generated training data is selected from compatible types according to the dataset-construction rules and is not a measured psychometric result for every original candidate. In addition, a single combination of subject code, score, and Holland type may be compatible with several major groups. These

valid alternatives are stored as separate multiclass rows, so identical predictor values can occur with different labels and create unavoidable ambiguity for a single-label classifier.

The processed CSV files do not retain CandidateID and SourceYear. Consequently, the current evaluation cannot guarantee that records originating from the same candidate are isolated in one fold, and the high frequency of repeated discrete patterns may make row-level cross-validation optimistic. Weighted metrics are also influenced mainly by common classes and may hide weak performance on rare major groups. The 2026 dataset illustrates this issue because several classes contain very few observations.

The Hybrid RAG component depends on the coverage and freshness of the curated knowledge sources and on the availability of vector, graph, reranking, and language-model services. Retrieval was evaluated on 60 questions across four configurations, but the set emphasizes representative and prominent disciplines rather than assigning a separate question to every CTU major. The two out-of-scope cases were excluded from retrieval averages, and the retrieval-only run did not measure generated-answer faithfulness, relevance, correctness, or citation accuracy. Generated explanations should therefore still be checked against current official admission information, and broader user and end-to-end evaluation remains necessary.

4.3. Future Development

Future work should collect additional real-user Holland assessment data and more examples for rare major groups. CandidateID and SourceYear should be retained so that candidate-grouped and year-based holdout evaluation can be performed. Macro F1-score, balanced accuracy, per-class results, probability calibration, and an independent test set should be reported together with weighted metrics. Because several majors may be valid for one input, multilabel classification or learning-to-rank methods should also be investigated.

The knowledge base should be expanded using regularly updated official sources, with versioning and automated checks for outdated or unavailable links. Retrieval evaluation should measure whether the correct evidence is found, while answer evaluation should examine faithfulness, relevance, completeness, and citation accuracy. The prototype should also undergo broader user testing, performance testing, security review, accessibility improvement, deployment monitoring, and interface refinement before it is used beyond an academic demonstration.

REFERENCES

[1] E. Bullock-Yowell and R. C. Reardon, Holland's RIASEC Hexagon: A Paradigm for Life and Work Decisions. Tallahassee, FL, USA: Florida State Open Publishing, 2024. doi: 10.33009/fsop_bullock-yowell0524. [Online].

Available: https://files.eric.ed.gov/fulltext/ED653445.pdf.

[2] Scikit-learn Developers, "Multiclass and multioutput algorithms," scikit-learn User Guide. [Online].

Available: https://scikit-learn.org/stable/modules/multiclass.html#multiclass-classification.

[3] Scikit-learn Developers, "Logistic regression," scikit-learn User Guide. [Online].

Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression.

[4] Scikit-learn Developers, "Random forests," scikit-learn User Guide. [Online].

Available: https://scikit-learn.org/stable/modules/ensemble.html#random-forests.

[5] Scikit-learn Developers, "Gradient-boosted trees," scikit-learn User Guide. [Online].

Available: https://scikit-learn.org/stable/modules/ensemble.html#gradient-boosted-trees.

[6] Scikit-learn Developers, "Support vector machines," scikit-learn User Guide. [Online].

Available: https://scikit-learn.org/stable/modules/svm.html.

[7] Scikit-learn Developers, "Cross-validation: evaluating estimator performance," scikit-learn User Guide. [Online].

Available: https://scikit-learn.org/stable/modules/cross_validation.html#k-fold.

[8] A. Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems. Sebastopol, CA, USA: O'Reilly Media, 2025. [Offline].

Available:https://www.amazon.com/Hands-Machine-Learning-Scikit-Learn-PyTorch/dp/B0F2SG98Q9.

[9] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online].

Available: https://arxiv.org/abs/2005.11401.

[10] P. P. Lan, "RAG," course material, Department of Software Engineering, College of Information and Communication Technology, Can Tho University, unpublished. [Offline].

[11] V. Nguyen, "Thực hành huấn luyện mô hình Machine Learning với Scikit-learn", YouTube video. [Online].

Available: https://www.youtube.com/watch?v=fm8nn374caw&t=5025s.

[12] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," 2022. [Online].

Available: https://arxiv.org/abs/2212.10496.

[13] J. Chen et al., "M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," 2024. [Online].

Available: https://arxiv.org/abs/2402.03216.

[14] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in Proc. 32nd Int. ACM SIGIR Conf., 2009. [Online].

Available: https://dl.acm.org/doi/10.1145/1571941.1572114

[15] "The Complete Guide to Hybrid Search in RAG (BM25 + Embeddings + Reranker)", YouTube video. [Online].

Available: https://www.youtube.com/watch?v=XvKiTfd6Xvo.

[16] "Neo4j in 100 Seconds", YouTube video. [Online].

Available: https://www.youtube.com/watch?v=T6L9EoBy8Zk.

[17] Vite, "Getting Started", Vite Guide. [Online].

Available: https://vite.dev/guide/.

[18] daisyUI, "Install daisyUI as a Tailwind plugin." [Online].

Available: https://daisyui.com/docs/install/.

[19] "FastAPI tutorial", YouTube video. [Online].

Available: https://www.youtube.com/watch?v=-IaCV5-mlSk.

[20] S. Ramírez and contributors, "FastAPI documentation." [Online].

Available: https://fastapi.tiangolo.com/.

[21] OpenAI, "OpenAI API Platform Documentation." [Online].

Available: https://developers.openai.com/api/docs.

[22] OpenAI, "GPT-5.6 Sol Model." [Online].

Available: https://developers.openai.com/api/docs/models/gpt-5.6-sol.

[23] Google, "Gemini 3.1 Flash-Lite," Gemini API Documentation. [Online].

Available: https://ai.google.dev/gemini-api/docs/models/gemini-3.1-flash-lite?hl=vi.

[24] Chroma, "Introduction," Chroma Documentation. [Online].

Available: https://docs.trychroma.com/docs/overview/introduction.

[25] Neo4j, "Graph database concepts," Neo4j Documentation. [Online].

Available: https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/.

[26] anhdung98, "Dữ liệu thô Điểm thi tốt nghiệp THPT 2025," GitHub repository, 2025. [Online].

Available: https://github.com/anhdung98/diem_thi_2025.

[27] NamBZ, "Tool Crawl Điểm Thi THPT 2026," GitHub repository, 2026. [Online].

Available: https://github.com/NamBZ/diemthi-thpt-2026.

[28] "Dữ liệu điểm thi từ báo Tuổi Trẻ (2026)," MediaFire dataset. [Online].

Available: https://www.mediafire.com/file/1zicb03hkxlwzr7/diem_thi_tuoitre.csv/file.

[29] H. Phạm, "Các khối thi đại học và xu hướng ngành nghề hot nhất năm 2025 dành cho học sinh 2k7," FPT Shop. [Online].

Available: https://fptshop.com.vn/tin-tuc/danh-gia/cac-khoi-thi-dai-hoc-173774.

[30] Can Tho University, "Danh mục ngành, chỉ tiêu, tổ hợp xét tuyển đại học chính quy năm 2026." [Online].

Available: https://tuyensinh.ctu.edu.vn/chuong-trinh-dai-tra/841-danh-muc-nganh-va-chi-tieu-tuyen-sinh-dhcq.html.

[31] Can Tho University, "Tuyển sinh Đại học Cần Thơ." [Online].

Available: https://tuyensinh.ctu.edu.vn/

[32] OpenAI, "ChatGPT desktop app," ChatGPT Learn. [Online].

Available: https://learn.chatgpt.com/docs/app/

[33] N. H. Thi, "Tìm ngành, trường đại học hợp sở thích của bạn | UniPath", Unipath. [Online].

Available: https://unipath.vn/

[34] LlamaIndex, "Evaluating," LlamaIndex Documentation. [Online].

Available: https://docs.llamaindex.ai/en/v0.10.19/understanding/evaluating/evaluating.html

APPENDICES

APPENDIX A: DECLARATION OF AI-ASSISTED USE

Generative AI tools, including ChatGPT, were used as supporting aids during research and implementation. Their purposes were to help explain frontend page-routing and parameter-passing logic, suggest backend structure and syntax based on my ideas, assist in interpreting model-training results and observed limitations, and support the search and organization of public information used in dataset construction. [32]

AI was not used to randomly fabricate examination scores or target labels. The examination-score data were derived from public 2025 and 2026 sources. The processed datasets were produced programmatically by combining those records with subject-combination, admission, major-group, and Holland mapping rules. Reservoir sampling selected records across the input streams to control the maximum size of each class; this sampling did not create synthetic examination scores. [26], [27], [28], [29], [30]

All project ideas, architecture decisions, dataset-generation rules, implementation choices, verification activities, and final conclusions remain my responsibility. AI-generated suggestions were reviewed and adapted before they were used in the project or report.

APPENDIX B: EVALUATION EVIDENCE

Detailed report on the results of Model comparisons (include chart, table, confusion matrix):

https://docs.google.com/document/d/16RPklNWegEUlaECY9esXzWBV50BntAp4/edit?usp=sharing&ouid=117870117085908575830&rtpof=true&sd=true

RAG Evaluation Test Cases:

https://drive.google.com/file/d/1X3JyRIq4I_oBjxlVov__7MP-HDA8cBXy/view?usp=sharing

RAG Evaluation Results:

https://drive.google.com/file/d/1WeSwhvhRYVP3GSTKTg61KKYzv_JofTBL/view?usp=sharing

APPENDIX C: PROJECT SOURCE CODE REPOSITORIES

The complete source code of the project is publicly available in the following GitHub repositories:

Frontend source code:

https://github.com/tuanduy35399/client_NLCS.git

Backend source code:

https://github.com/tuanduy35399/backend_NLCS.git

Learn more about the author of this report:

https://github.com/tuanduy35399

Project Documentation Folder:

https://drive.google.com/drive/folders/1qq1npU0-gsDBgcHELsw3boBUi1ASctkm?usp=drive_link